ILLUSTRATED LEARNING BOOK | VOLUME 2

Computing Made Understandable

Data Structures



Original manga illustration: a story analogy. Formal labeled diagrams inside the book carry the technical definitions.

Organize information around the operations it must support. Follow every slot, link, stack, queue, branch and route.

First-year learning edition | 6 November 2026

K.I.V.A.S. Learning Lab | Foundations of Computer Science & Artificial Intelligence

Inside this book

Chapters are separate and numbered within each volume. Printed page numbers match this PDF. The PDF outline also includes teaching sections.

How to learn from this volume 3

Why Data Organization Matters 4

Arrays and Dynamic Sequences 9

Linked Lists 18

Stacks 26

Queues 33

Trees and Hierarchies 38

Graphs and Route Models 47

Choosing and Combining Data Structures 56

Glossary and retrieval cues 62

Reading, sources and credits 64

How to learn from this volume

You do not need to know a programming language. Begin with the story, predict a result, then trace the formal example. Cover the feedback until you have written a reason. A diagram is useful when you can explain what each box, arrow and number means.

Learning move	What to do	Evidence to keep
Predict	Write the next state before reading it	Your first answer
Trace	Apply one operation at a time	Intermediate states
Explain	Name the rule that justifies the change	A sentence or labeled diagram
Transfer	Change one input or boundary case	A new trace
Retrieve	Return later without the solution	A corrected explanation

Conventions used throughout the set

Indices start at zero. Arrays use length for current entries and capacity for reserved slots. Binary search uses inclusive low and high bounds and the lower midpoint. A missing result is distinct from index zero. Tree height counts edges. Campus graph edges are two-way unless explicitly stated. ReLU(z)=max(0, z); the step rule includes zero when written z> =0. A raw neural score has only the meaning the task gives it. Illustrative data are not reported experiments.

Your route

If variables and traces are new, read Volume I, Chapters 1 to 3 first. This volume begins with records and operations, then builds arrays, linked lists, stacks, queues, trees and graphs. Finish by choosing structures together.

A USEFUL STUDY RHYTHM

Try one or two sections in a 25-40 minute session. Finish with a small new example. After a day, repeat the chapter lab without looking at its solution. The separate Test Bank supplies unfamiliar cases, and the Learning Materials companion gives optional visual and interactive routes.

Artwork follows the supplied purple, black, white and violet theme. Manga scenes are narrative analogies. Labeled technical figures and worked tables define the algorithms and numerical examples. Essential information is also stated in text.

DS-CH01 / CHAPTER 01

Why Data Organization Matters

BY THE END

Describe records, keys, operations and invariants before choosing storage.

Algorithms operate on information. This chapter asks how organizing that information changes what a procedure can do efficiently.



ART-DS-CH01 Numbered slots, stacks, queues and branching shelves suggest different ways to organize information. These library analogies introduce the formal structures that follow.

1.1 The same facts, different questions

Imagine that Pip pours twenty book records into a box. All the information is present, but finding one title still means lifting cards until it appears. Put the same cards into numbered slots and a known slot becomes easy to reach. Sort them by copy ID and comparisons can rule out regions. The information and its organization are separate design choices.

A data structure organizes stored information together with rules for accessing and changing it. A record groups facts about one thing; a field is one part of that record. For a book copy, fields might be copy ID, title and availability. A key is a field used to identify or compare records. A lasting copy ID differs from an array index, which describes where a record currently sits. Moving a card does not create a new physical book.

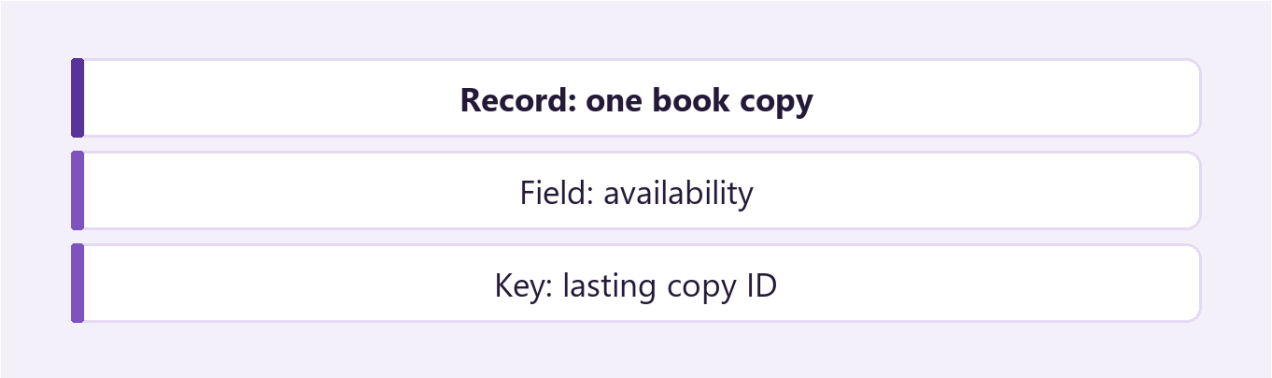


FIG-DS-01-01 Read the boxes as different levels of description, not as three storage containers. One record contains fields; one selected field can serve as its key.

Investigate and explain

Make records for copies 12, 24 and 37, then arrange them in reverse order. Before looking below, identify which facts change and which facts remain the same. Explain why a receipt should record a copy ID rather than only a slot number.

Copy ID	Old index	New index	Physical identity
12	0	2	Same copy
24	1	1	Same copy
37	2	0	Same copy

Reading the result: The positions of 12 and 37 change, but the copies do not. An index is meaningful relative to one current arrangement. If a later insertion shifts records, an old index may name a different item. A receipt needs a durable identifier so that later staff can still resolve which copy was handed over. This distinction will recur in array deletion and the final library project.

Try it yourself: Two copies have the same title. Must their keys be equal?

Check your reasoning: No. Titles can repeat while unique copy IDs distinguish physical copies. If the task is grouping titles, title can be a comparison key without being a unique identifier.

1.2 Choose the operations before the container

The librarian has three questions: which book is at index 4, which request arrived first, and which campus rooms connect? One neat-looking arrangement cannot answer all three equally well. Start by naming the operation: read at a position, remove the oldest waiting item, or follow connections. Then ask what facts must remain true after every change.

An interface states what operations mean, such as enqueue adding a request at the rear and dequeue removing the front. An implementation is the storage and procedure that fulfil that

promise. The same queue interface can be implemented with links or with a circular array. An invariant is a condition that every permitted operation preserves. For a basic queue, waiting requests retain arrival order. An implementation is acceptable only if its behavior matches the interface on ordinary and boundary cases.

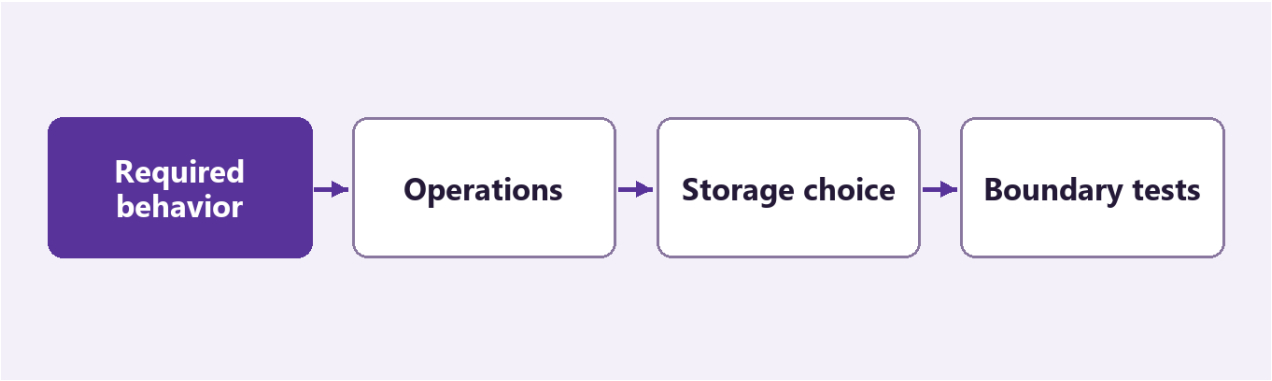


FIG-DS-01-02 The arrows describe a design process. First decide the promise a user relies on, then choose a representation and test that promise.

Investigate and explain

A help desk accepts A, then B, then C. One clerk starts the oldest waiting request. Draw the queue before and after the first removal. Then describe how the test would expose an implementation that accidentally removes the newest request.

Moment	Waiting order front to rear	Next removal
After A	A	A
After B	A, B	A
After C	A, B, C	A
After one removal	B, C	B

Reading the result: A removal of C would violate FIFO even if the data structure still contained two valid requests. Correctness includes the promised order, not only the number of records remaining. To test an empty queue, specify an explicit empty result or error; do not silently invent a request. To test a one-item queue, remove the sole item and verify that both its content and empty-state bookkeeping are correct.

Try it yourself: Must callers know whether a queue uses links?

Check your reasoning: Usually no. They need its behavior and relevant performance limits. Hiding storage details lets the implementation change while preserving the same operation contract.

1.3 A cost needs a question and a unit

Suppose a worker can reach a numbered slot directly, while another follows a chain from the first card. Both may fetch the fourth record, but the work they perform differs. Calling one structure fast without naming the operation hides the important decision. The same structure can support cheap reading and costly insertion.

In this book, compare simple operations such as reads, comparisons, shifts and reference updates. Let n mean the number of current records. A fixed-size change is O(1); a traversal that can visit every record is O(n). These are growth descriptions under a stated model, not stopwatch promises. Include locating the edit position: inserting after a supplied node reference differs from first searching for that node by value.

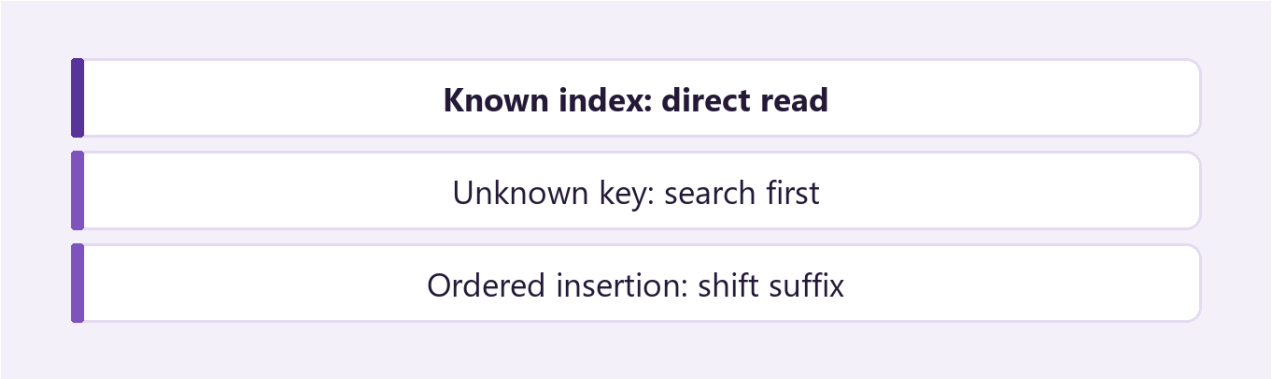


FIG-DS-01-03 Each panel names information the caller already has. A performance claim changes when that starting information changes.

Investigate and explain

For six cards A through F, count existing records moved when inserting X immediately after D in an array with one spare slot. Compare that with inserting X after a supplied linked node D. Keep finding D separate from changing the structure.

Representation	Finding position	Structural edit
Array	Index 3 supplied	Move F then E, write X
Singly linked list	Reference D supplied	Set X.next to E, D.next to X
Linked list by value	Inspect A, B, C, D	Then the same two link writes

Reading the result: The array shifts two existing records, then writes the new one. The linked edit changes two references, once D is known. Neither result proves that one representation is universally superior. Arrays give useful positional access; linked nodes spend space on references. Real caches and allocation policies also matter, but the operation model is enough for the design comparisons here.

Try it yourself: Does spare array capacity make front insertion constant time?

Check your reasoning: No. Preserving order still moves all n existing entries one position right. Spare capacity avoids an immediate resize; it does not avoid the shifts.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Design three records for a library copy, a waiting request and a route edge. Identify a durable key for the first two and the operation each record supports. Explain why an array position should not be a permanent copy identity.

Worked guidance

A copy record can hold copy ID, title, shelf and availability. A request record needs its own request ID, requested copy ID, destination and status. A route edge records endpoints, cost and current access. Indices can change when storage is rearranged, while durable identifiers continue to name the same entity.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can describe records, keys, operations and invariants before choosing storage. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

DS-CH02 / CHAPTER 02

Arrays and Dynamic Sequences

BY THE END

Read and update by index, preserve order while shifting, and distinguish length from capacity.

Equal-width slots make a known index easy to reach. Keeping an ordered row packed can require shifting a suffix. A growing array occasionally has to copy its contents.

2.1 A wall of numbered lockers

Pip faces six identical lockers in a straight line. Their labels begin at zero. “Open locker 4” sends it to a particular door; “find the locker containing Atlas” might send it to all six. An array gives a sequence of slots an index-based organization.

In a simple memory model equal-width slots occupy consecutive addresses. Given a base address and the width of one slot, the address of index i is base + i × width. A slot might itself hold a reference to a larger object elsewhere. The slots still have equal widths in this model.



FIG-DS-02-01 Index 4 is the fifth door. Counting the index as an offset from the first door explains why the first offset is zero.

Look closer: One locker can hold a card with directions to a large book elsewhere. Equal-width array slots need not imply identical-size physical books. The direct calculation locates the slot containing the direction.

Calculate a slot location

Suppose the first slot begins at address 100 and each occupies four address units. Compute the starting address of every second slot, then check the final valid index.

Index	Offset	Address
0	0 × 4	100
2	2 × 4	108
4	4 × 4	116
5	5 × 4	120

Reading the result: Calculating 116 uses the same number of arithmetic steps as calculating 108. A large index does not require visiting the intervening slots. This explains constant-time access in the usual model for a valid known index.

Try it yourself: Would address 124 be a seventh legal locker in this six-slot array?

Check your reasoning: No. The formula can produce an address beyond the reserved region. A bounds check must reject index 6 before any read.

Another case: Read an address backward

Slots start at 200 and have width 8. One begins at 224. Which index does it represent? Could 222 be the starting address of a slot under the same layout?

Feedback: Subtract the base: 224 - 200 = 24. Three widths of eight make 24, so the index is 3. The offset 22 for address 222 is not a multiple of eight. It might lie inside a slot, but it is not the start of one.

2.2 A walk past every locker

A clerk wants the sum of the loan counts in four lockers. Knowing the position of one count will not reveal the others. Traversal visits the entries systematically. Here the index advances from zero through the final valid position, while a running total preserves the work already done.

Reading A[i] observes its current value. It does not remove the value or move any later entries. Draw the reading finger separately from the cards. Otherwise it is easy to mistake movement of the algorithm for movement of the data.

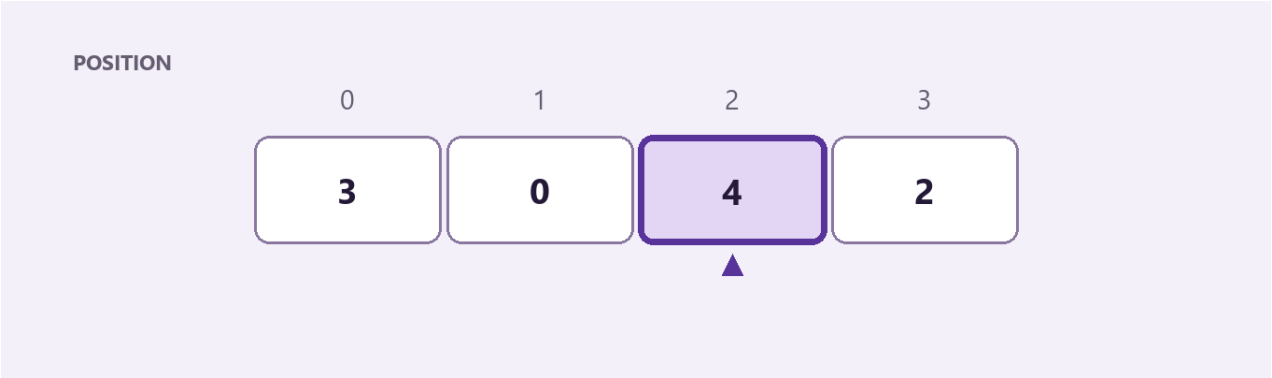


FIG-DS-02-02 The reading finger has reached the third slot, value 4. Zero is a valid stored count at index 1, so traversal must pass it rather than treating it as an end marker.

Look closer: Track whether an instruction reads or changes the array. Adding A[i] into total writes total while leaving A[i] alone. It is possible to finish a long computation with the input array exactly as it started.

Read an array without changing it

Trace the sum on [3, 0, 4, 2]. First calculate the total by hand. Next record the index and total after each visit. Explain why “stop at zero” gives a different result.

Index	Value read	Total after
0	3	3
1	0	3
2	4	7
3	2	9

Reading the result: The correct total is nine. Stopping at zero reports three and never visits the last two slots. A collection's length provides a reliable stopping point when any ordinary value, including zero, might be data.

Try it yourself: Can the same traversal find the number of occupied lockers?

Check your reasoning: Yes, if an occupancy rule is specified. If positive counts mean occupied, increment a tally once for each value greater than zero; the result here is three.

Another case: A maximum needs a different start

Pip wants the largest reading in [-4, -2, -7]. A helper sets best to zero and updates it only for a larger reading. Trace the result and repair it for nonempty input.

Feedback: The helper returns zero, a value absent from the row. Initialize best from the first reading, then inspect the rest. The trace updates -4 to -2 and retains -2 on the final reading. An empty row needs a separately specified no-reading result.

2.3 Changing a value in place

A book has moved from shelf C to shelf F. Its catalogue record still occupies the same slot. An update replaces a value at an existing index. The sequence length and the later indices stay the same. A visitor who saved the index can still use it if the catalogue promises stable slots for this operation.

Inserting a new record means keeping the existing records as well. Replacing one would lose a book. Ask whether the collection should contain the same number of entries after the operation. This question distinguishes two instructions that can sound alike at the desk.

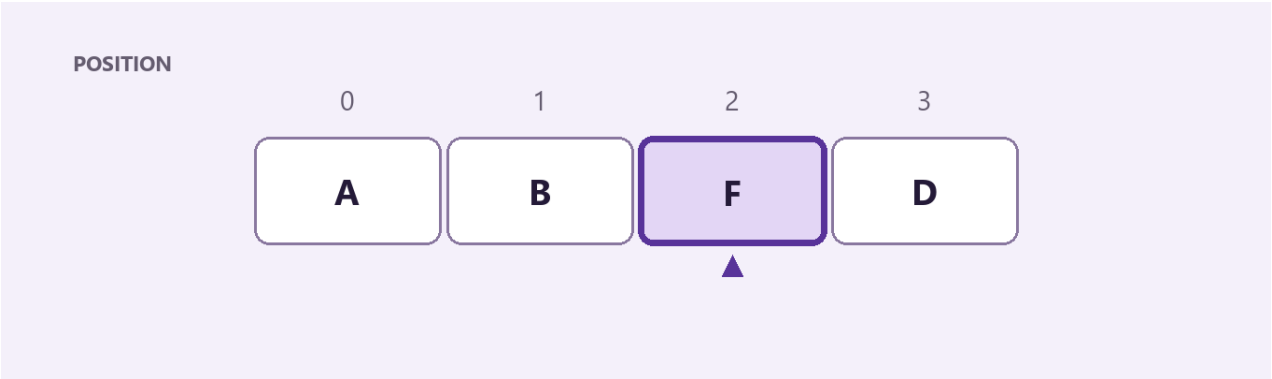


FIG-DS-02-03 The outlined shelf is the updated entry. Compare the four-slot result with the original [A, B, C, D]. Only one slot differs.

Look closer: Write IDs on the backs of the cards and shelves on their fronts. A shelf update changes a front label. This small exercise prevents an editing interface from confusing the identity of a record with one of its editable fields.

Distinguish replacement from insertion

Start with four shelf slips and write “replace slot 2 with F.” Perform that operation, then reset and simulate “insert F before slot 2.” Record the whole row and its length after both operations.

Action	Result	Length
Start	A B C D	4
Replace slot 2	A B F D	4
Insert before slot 2	A B F C D	5

Reading the result: An insertion shifts C and D one slot to the right if order must be preserved. The replacement can be done directly once the destination index is known. Both stories mention F and index 2, yet the work differs.

Try it yourself: What if the clerk knows only the title, not its index?

Check your reasoning: Locating the record may require a search before the update. A constant-time write does not make the entire find-and-update job constant time.

Another case: A copy changes shelf

Record 24 currently occupies array index 1 with shelf B4. Replace its shelf with C2 and then look it up by ID. Which values should still identify the same copy? Which field must change in the receipt?

Feedback: The ID remains 24 and the record still occupies index 1 for this replacement. The lookup should now report C2. If its answer is B4, investigate whether the lookup read an obsolete copy of the record rather than assuming the search strategy is faulty.

2.4 Make room before you write

Five items must fit into a six-slot rack with one spare position at the end. To insert a new card near the front, shift the suffix one place to the right. Start with its rightmost item so each write goes into a slot whose previous contents have already been saved or moved.

Distinguish length, the number of occupied entries, from capacity, the number of reserved slots. The spare slot is available for movement but lies outside the logical sequence until insertion completes. Increase the length after making room and writing the new card.

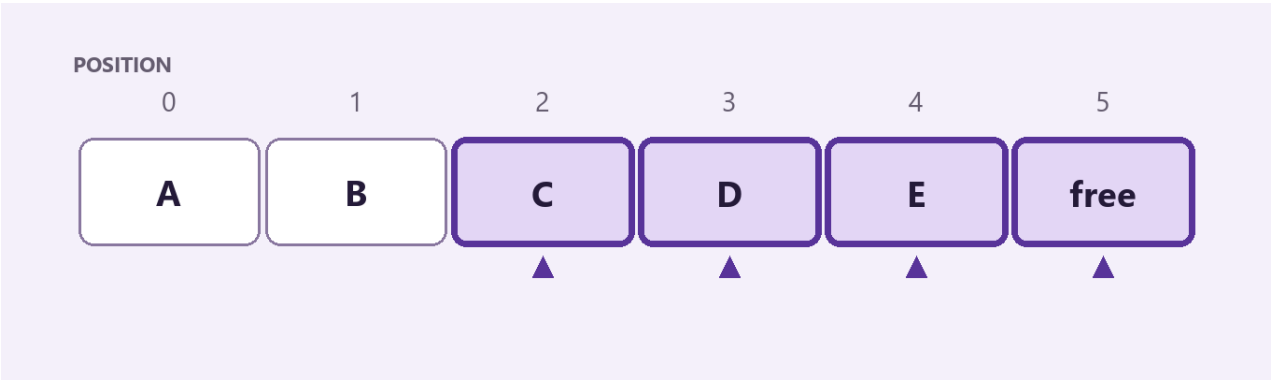


FIG-DS-02-04 To insert X at index 2, move E into the spare slot, then D and C one step right. The outlines mark the region used to make room.

Look closer: For n occupied entries and an insertion at position i, there are n - i existing entries from i onward. Those are the entries shifted in this packed ordered model, provided a spare final slot exists.

Shift a suffix without losing it

Write six boxes on paper and pencil in the five letters. Carry out each assignment without erasing a source until it is safe. Compare the steps below with a left-to-right shift.

Assignment	Slots after write
A[5] <- A[4]	A B C D E E
A[4] <- A[3]	A B C D D E
A[3] <- A[2]	A B C C D E
A[2] <- X	A B X C D E

Reading the result: Temporary duplicate values are expected while shifting. In a left-to-right shift, copying C into D's slot before saving D would cause a later step to copy C twice. The movement direction follows from which information must survive.

Try it yourself: How many existing entries move when appending at index 5 instead?

Check your reasoning: None when capacity is six and length is five. Write X into the spare slot and increase length. An already-full rack would require a different policy.

Another case: An insertion at the very front

Begin with [B, C, D] in a rack with one spare slot. Insert A at index 0 while preserving order. Write the destinations of the old records in the order they must be filled.

Feedback: D moves to index 3, C to 2, and B to 1. A can then be written at 0. Three existing records move. Starting at B and shifting right would destroy C unless it had been saved. Compare this with appending A at the end, which shifts none.

2.5 Close a gap after deletion

Removing a card from an ordered row leaves a hole. To keep the remaining records together in their original order, move the following entries one place left. This time work from left to right because the empty space starts on the left side of the moving suffix.

After shifting, reduce the logical length. A final unused slot may still contain stale bits or a stale reference; an implementation can clear it. Traversal should follow the new length. Deleting an item does not automatically make the reserved rack physically shorter.

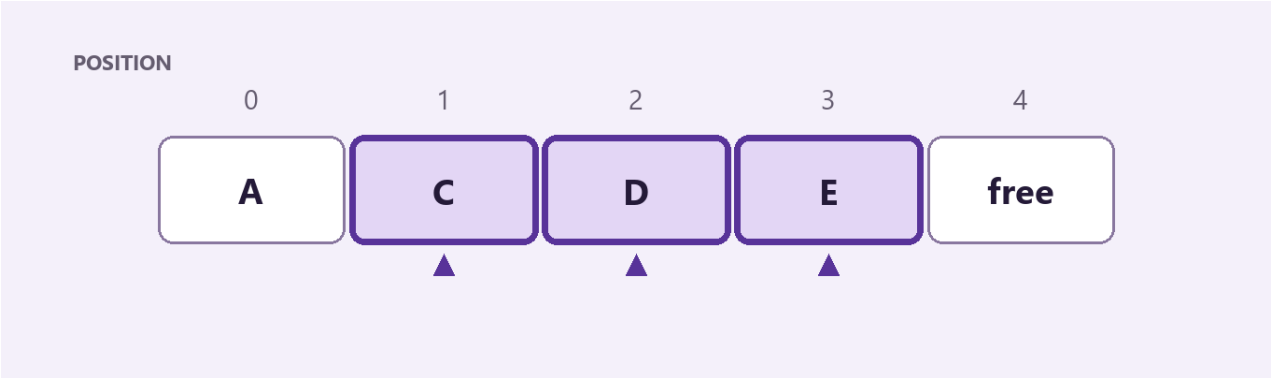


FIG-DS-02-05 B has been deleted from [A, B, C, D, E]. The outlines identify surviving records whose indices changed. Record C moved from index 2 to 1.

Look closer: Deletion at the final valid position has an empty suffix. Shortening the length suffices for the logical sequence. This is why saying that every deletion shifts the whole array overstates its cost.

Track an index that changes its meaning

Pip wrote down “slot 3 means D” before the deletion. Use an arrow for each copy, update the length, and test what a later read of slot 3 returns. Explain why a durable book identifier is useful.

Step	Logical or working row
Copy C left	A C C D E
Copy D left	A C D D E
Copy E left	A C D E E
Set length to 4	A C D E

Reading the result: Index 3 now contains E. An index describes a position in this version of the sequence. A lasting identifier stays with a book even if the catalogue reorganizes its positions. Confusing the two can send a visitor to the wrong record.

Try it yourself: If order can change, can deletion use fewer moves?

Check your reasoning: Copy the last occupied entry into the removed slot, then shorten the row. That uses at most one item move but can change the relative order required by a sorted catalogue.

Another case: A deletion at the boundary

Delete the only item Q from a capacity-four rack. Predict the new length and the legal indices for reading current items. Then append R without resizing. Which slot will R occupy?

Feedback: The new length is zero, with no legal occupied index. Capacity can remain four. Appending R writes index 0 and increases length to one. Residual data in unused slots must never be interpreted as active members of the sequence.

2.6 When the row fills up

The fifth parcel arrives for a four-slot rack. A fixed array has no valid fifth slot. A dynamic array can present a growing sequence by allocating a larger backing array, copying existing entries, and switching to the new storage. Some appends are consequently much more expensive than others.

If capacity doubles whenever full, many cheap appends occur between expensive copies. Across a long sequence the average cost per append can remain bounded under this growth policy. That is an amortized cost, a guarantee about the total work of a sequence, not a promise that every individual append is cheap.

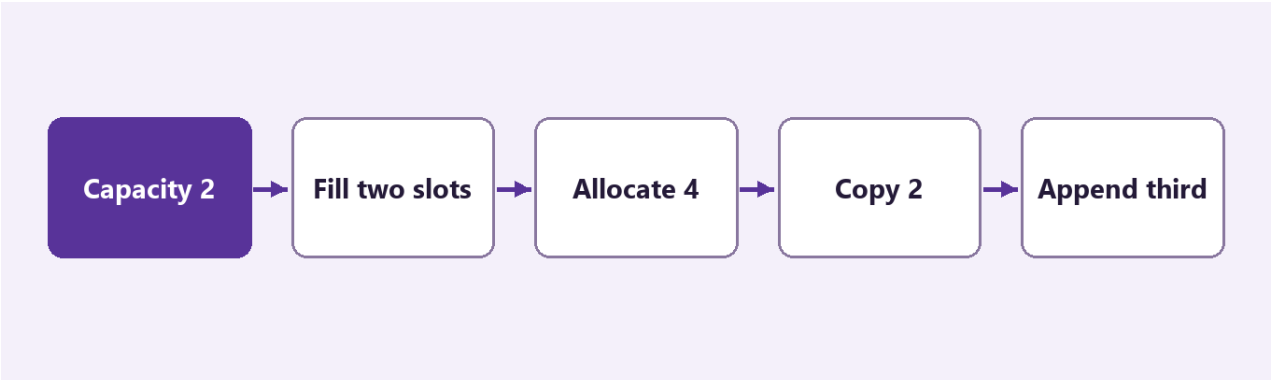


FIG-DS-02-06 A resize must retain both earlier entries and install the new one. The expanded rack has one spare slot immediately after the third insertion.

Look closer: Put an outline around the active portion of a rack after each append. A resize from capacity four to eight creates four spare slots before the new write, then three after it. Capacity and free space have different values.

Choose a capacity policy

Start with capacity two and zero entries. Append A through F. Predict which appends trigger growth. Count only copies of old entries in the copy column and separately count each newly written item.

Append	Capacity after	Old entries copied
A, B	2	0
C	4	2
D	4	0
E	8	4
F	8	0

Reading the result: There are six old-entry copies and six new-item writes across six appends. The append of E alone copies four existing entries. A designer with a strict deadline for each operation might reserve capacity in advance. Knowing a maximum size makes that possible.

Try it yourself: What if it is hard to reserve a long consecutive row at all?

Check your reasoning: Separate containers connected by links can represent a sequence without requiring neighbouring slots. Volume II, Chapter 3 examines the trade-off in finding a position.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Create [A, B, C, D] with capacity six. Insert X at index 1, delete the record at index 3, and append Y. Show every shift and distinguish current length from capacity.

Worked guidance

Insertion shifts D, C, B right and yields [A, X, B, C, D]. Deleting index 3 removes C and shifts D left, giving [A, X, B, D]. Append Y gives [A, X, B, D, Y], length five, capacity six. Slot five is spare, not a current item.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can read and update by index, preserve order while shifting, and distinguish length from capacity. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

DS-CH03 / CHAPTER 03

Linked Lists

BY THE END

Traverse from a head reference, splice and bypass nodes, and include the cost of locating the edit.

Links encode sequence independently of physical position. Local editing can be cheap once the relevant references are available. Preserve the old successor before rewiring.

3.1 A treasure hunt through scattered boxes

Pip finds a note reading “start at locker 40.” Inside is a book and a second note reading “next: locker 12.” The books form a sequence even though their physical locker numbers jump around. To find the next book, Pip follows the stored instruction.

A linked-list node contains a value and a reference to another node. A reference identifies where a node can be reached. The head reference identifies the first node. In a singly linked list each node has one next reference. A special value called null marks the absence of a next node.

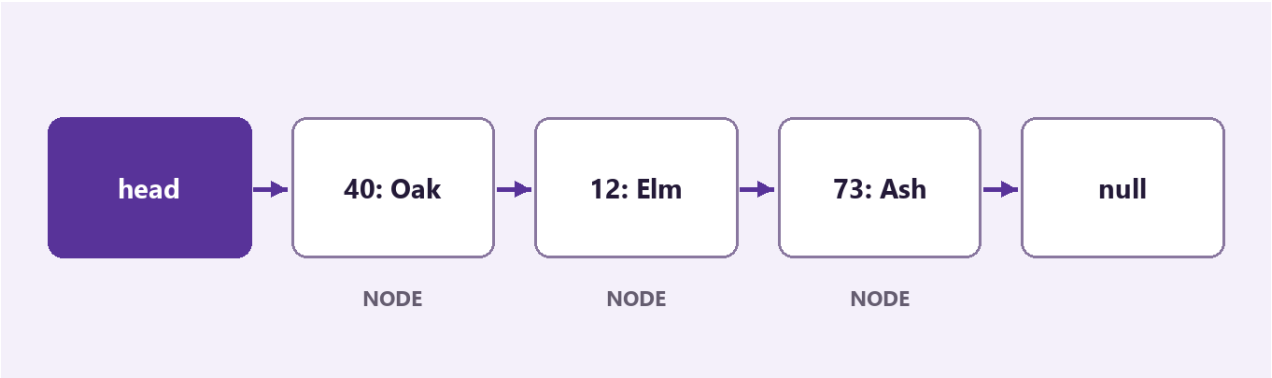


FIG-DS-03-01 Follow the arrows, rather than sorting the locker numbers. The values are Oak, Elm, Ash in that order. The null marker is a stopping signal and holds no book.

Look closer: The head is a reference, not a dummy book. An empty list can be represented with head equal to null. Including an imaginary first data node in every drawing causes off-by-one errors during traversal.

Follow the label inside each node

Write the following records on separate slips and scatter them. Start at head 40. Record the value in your hand and the label of the slip to pick up next. Consider which information survives shuffling the slips.

Node label	Value	Next
12	Elm	73
73	Ash	null
40	Oak	12

Reading the result: The physical order of the slips on your desk does not determine their logical order. Following the reference 40, then 12, then 73 reconstructs the sequence. If the head were lost, a reader would no longer have its designated starting point.

Try it yourself: Does adding one to node label 40 locate its successor?

Check your reasoning: No. The next label is stored inside node 40 and happens to be 12. Consecutive numeric addresses are unnecessary.

Another case: A node label that resembles a value

Suppose node 12 contains the value 40 and next reference 7. Node 40 contains a different book. Which node should Pip visit next from node 12? Which piece of the record answered your question?

Feedback: Pip follows next to node 7. The stored value 40 could be a book number unrelated to node 40's address. Giving the value and next fields distinct boxes prevents an accidental jump to a node whose label merely coincides with a data value.

3.2 Walking without losing the start

The desk needs to inspect every linked node while preserving access to the first one. Copy head into a temporary reference called current. Reading or changing current does not by itself rewrite the stored links. Moving head instead could discard the normal entry point into the list.

WHILE current is not null, read current's value, then set current to its next reference. Test for null before reading. On an empty list head is null at the start, so there is no node to inspect. This traversal assumes the next chain eventually reaches null.

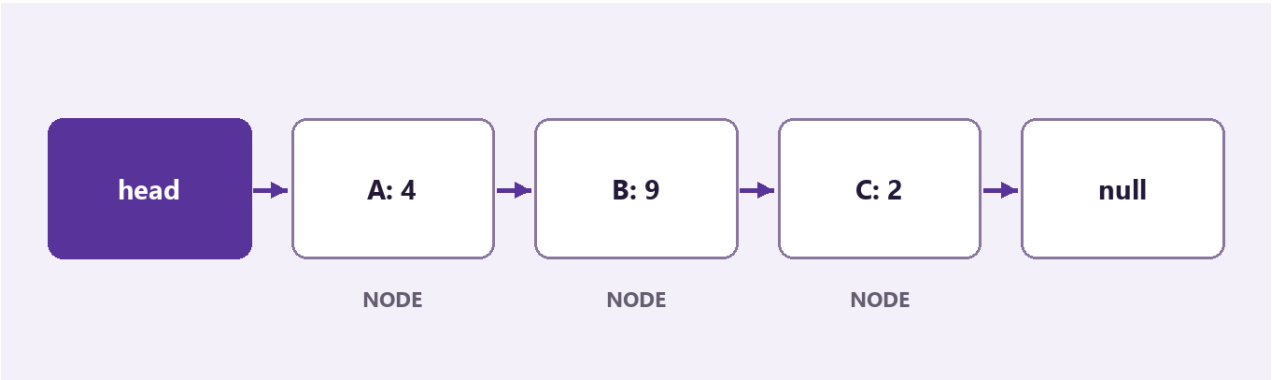


FIG-DS-03-02 A, B and C name nodes. The numbers inside are their data. Looking for a missing 7 follows the links until current becomes null.

Look closer: A traversal normally keeps the structure intact. If you replace current.next instead of current, the search itself rewrites a link. Draw pointer movement with a movable finger and link writes with an eraser.

Find a value with a temporary pointer

Trace a search for 9, then for 7, using the same three nodes. Write down the visited node labels. For a successful search stop on equality; for failure show the state immediately after the last real node.

Target	Visited nodes	Result
9	A, B	Reference B
7	A, B, C	MISSING after null
Any target, empty list	None	MISSING immediately

Reading the result: A target's numerical size cannot tell Pip where its node is stored. Even the third position requires following two next references from head. A singly linked list generally lacks constant-time indexed access from the head.

Try it yourself: What would happen if C.next pointed to A during the search for 7?

Check your reasoning: The traversal would cycle. Its promise to terminate depended on an acyclic chain. A drawing that includes a loop describes a different structure from this ordinary list.

Another case: Make a reversed head harmless

A student writes current <- head.next before checking whether head is null. Try empty, one-node and two-node chains. Identify both a possible invalid read and a valid node the procedure skips.

Feedback: On the empty chain there is no node at head whose next field can be read. On a nonempty chain it starts with the second node or null and skips the first value. Initialize current to head, then test current before inspecting its fields.

3.3 Splice a node into the chain

Suppose Pip already holds the reference to node A, whose successor is C. A new node B belongs between them. Only two next fields need writing. No later node has to move. This local advantage depends on having the predecessor, A, already in hand.

First point B.next at A.next. Then set A.next to B. If you change A.next first and subsequently copy it into B.next, B points to itself. Drawing arrows in the order you write them makes this failure surprisingly easy to see.

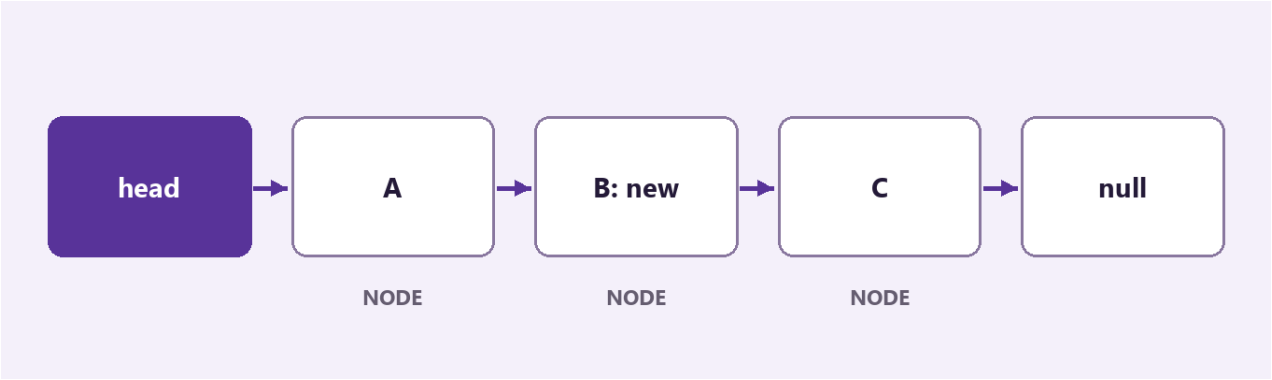


FIG-DS-03-03 The final chain visits B exactly once before continuing to C. The arrow from B to C must preserve the destination of the old arrow from A.

Look closer: You can check a splice without knowing all the node values. Before and after the edit, follow the path from A to its old successor. The new path should pass through B exactly once.

Save the old successor first

Use two lengths of string or drawn arrows. Trace the correct two writes and a reversed version. After each write, start from head and record which nodes are reachable. B is available separately until linked in.

Stage	A.next	B.next
Initial	C	null
Save successor	C	C
Connect A to B	B	C
Faulty reversed finish	B	B

Reading the result: The correct final path is A, B, C. The faulty version cycles at B and loses C from the head chain. A picture of the final intended list is insufficient to establish that the sequence of writes builds it.

Try it yourself: How can B be inserted at the very front?

Check your reasoning: Set B.next to the old head, then make head refer to B. This also works for an empty list because the old head is null.

Another case: Append using a known tail

The known tail is T and T.next is null. You have a fresh node U. Adapt the two-write splice, then say which extra reference needs updating if the desk keeps a tail label.

Feedback: Set U.next to the old T.next, which is null, and T.next to U. Then update the separate tail label to U. Leaving tail at T would preserve traversal from head but make the next operation that relies on the tail label incorrect.

3.4 Bypass a node without breaking the rest

To remove B from A -> B -> C, make A refer directly to C. The resulting head chain bypasses B. Removing a node from a list describes a change in reachability from its head. What happens to the storage for B depends on the language and any other references.

Deleting the first node is a different boundary case. No predecessor node exists, so update the head itself to the old head's successor. Before reading that successor, make sure a head node exists. For a singleton the new head is null.

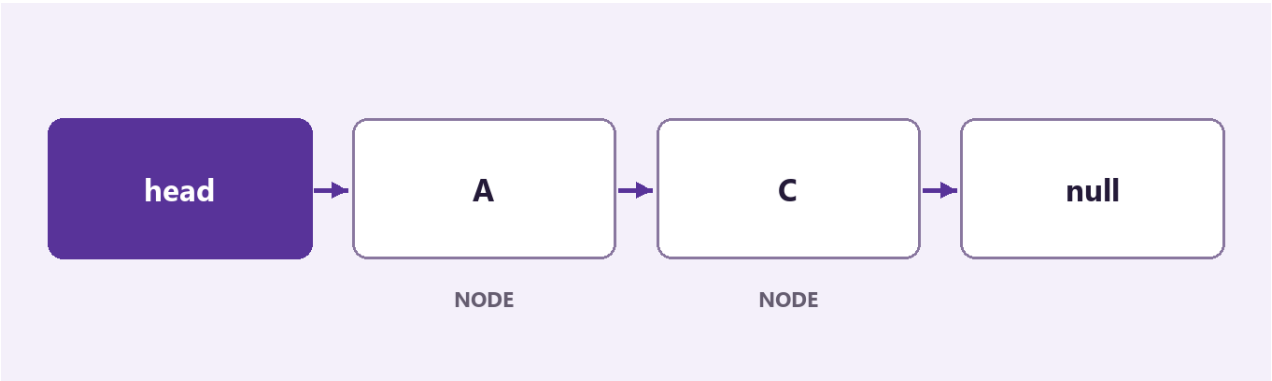


FIG-DS-03-04 After bypassing B, the arrow out of A leads to C. The value in C has stayed in its own node. No suffix was shifted as it would be in an ordered array.

Look closer: A removed node can still be referenced by another part of a program. Deleting a link from this list and freeing memory are distinct operations. For the paper model, test which nodes the head can now reach.

Delete the head and a middle item

Begin each case with a fresh chain. Predict exactly one reference field to change when the target and its predecessor, if any, are already known. Draw the resulting head path.

Initial	Delete	Write
A -> B -> C	B	A.next <- C
A -> B -> C	A	head <- B
A	A	head <- null
Empty	Any	Report missing

Reading the result: The same conceptual removal has two destinations for its write: head or a predecessor's next field. On an empty list there is no field inside a node to read. A safe procedure returns the agreed missing result.

Try it yourself: If a tail reference is maintained too, what must happen when deleting the only node?

Check your reasoning: Clear both head and tail. Leaving tail pointing at the removed node would make two descriptions of the same list disagree.

Another case: Remove a missing value safely

The chain A(4) -> B(6) -> C(8) ends in null. The request says remove value 7. Trace the search and name exactly which links may change if the contract promises to preserve the list on a miss.

Feedback: Visit 4, 6 and 8, then reach null. The output should say missing, and every stored link should still have its old destination. A procedure that deletes the last visited node would remove 8 without evidence that it was the target.

3.5 Count the journey to the edit

“Linked lists insert in constant time” is a tempting shorthand. It is accurate for a local splice after a known node under the usual model. A request such as “insert after book 77” may require scanning the whole list to locate that node first.

An array and a linked list can therefore trade places depending on what the caller knows. An index locates an array slot immediately, but preserving order may require shifts. A reference to a predecessor locates a linked edit immediately, while finding the predecessor by value may be costly.



FIG-DS-03-05 The comparison divides an edit into locating its place and changing storage. A saving in the second part matters only after the first part has been included.

Look closer: An array may answer "which record is fourth?" directly. A linked list may answer "which node comes after the one in my hand?" directly. Both give fast answers to particular questions; they are different questions.

Compare two complete insertion jobs

Build a six-entry chain and an array with one spare slot. Insert X after the fourth entry, first when handed a reference to that node, then when handed only its unique value. Count node inspections, existing-item shifts and next-field writes separately.

Scenario	Locate	Modify
List, known node	Already in hand	2 next writes
List, given value	4 inspections	2 next writes
Array, given index 3	Direct access	Shift 2 existing entries

Reading the result: The linked splice performs two next writes in both list cases. The end-to-end work differs because only the second case required discovery. On an array the two shifted entries originally occupied indices 4 and 5. The new item belongs at index 4.

Try it yourself: What if the only requirement is to remove the most recently added item?

Check your reasoning: A restricted set of operations can simplify the design. Volume II, Chapters 4 and 5 introduces the stack, which describes that behaviour regardless of the storage used underneath.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Draw A -> B -> C -> null. Insert X after B, then remove B using its predecessor A. Trace references after each write and explain the danger of overwriting the only route to a suffix.

Worked guidance

First set X.next=C, then B.next=X. The chain is A, B, X, C. To remove B, set A.next=B.next, which now names X. Reversing an insertion without preserving the successor can make X point to itself and lose access to C. Empty and head edits need separate handling.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can traverse from a head reference, splice and bypass nodes, and include the cost of locating the edit. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

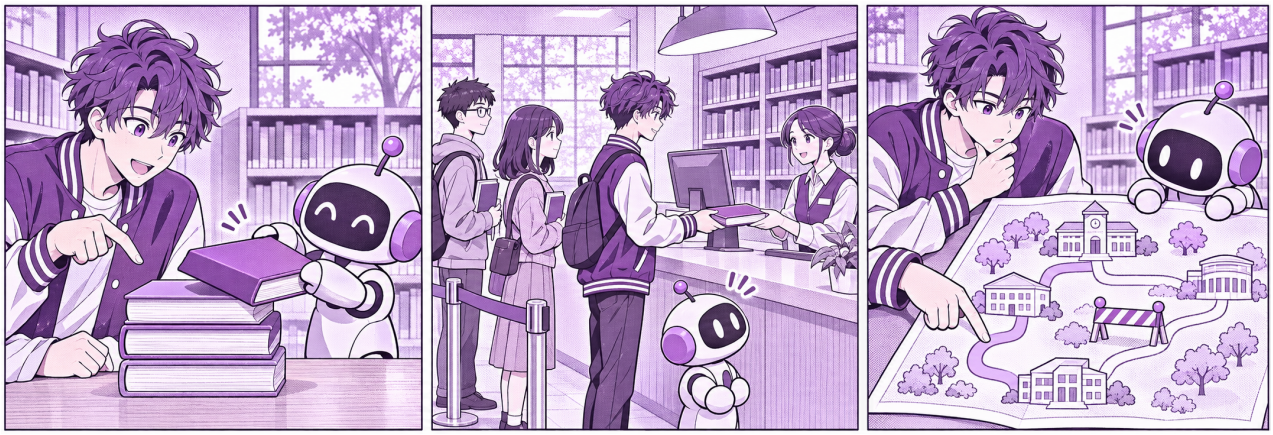
DS-CH04 / CHAPTER 04

Stacks

BY THE END

Trace push, pop and peek; design an undo record that preserves the information needed for reversal.

A stack is defined by the most-recent-first promise. Its storage can vary while that order stays the same.



ART-DS-CH04 The panels contrast a top-only stack, arrival-order service and a route map. Each structure supports a different question.

4.1 Choose which end is open

Three requests arrive at the desk in order A, B, C. Without asking what kind of service this is, can you say which request should leave first? A desk serving visitors in arrival order should remove A. A desk undoing its last three edits should remove C.

An interface states what operations are allowed and what results they produce. A stack admits and removes items at the same end. A queue admits them at one end and removes them at the other. Either can be implemented using suitable arrays or nodes. The rule about removal is what the visitor experiences.

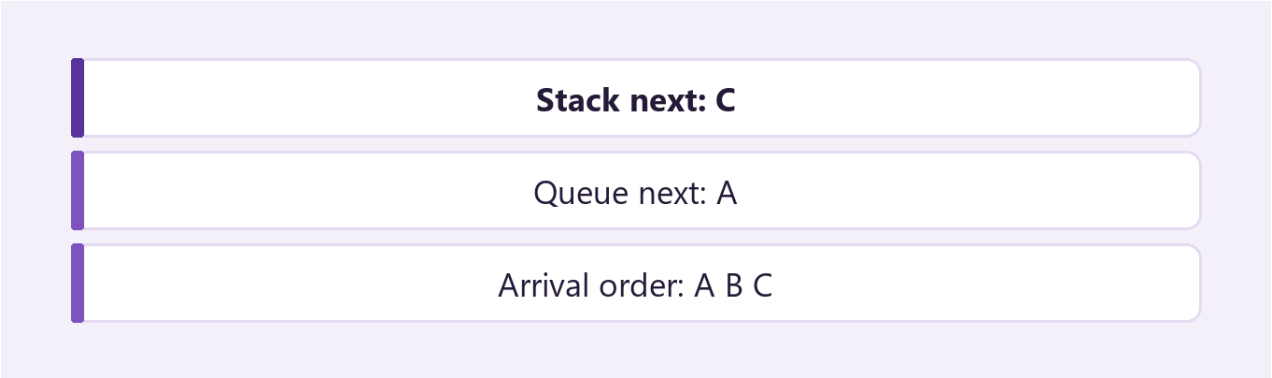


FIG-DS-04-01 Imagine three labelled tickets in arrival order. The two policies disagree at the very first removal even though the tickets and their arrival times are identical.

Look closer: Two ways to implement a queue can look quite different in memory. A visitor receiving service can test the interface by observing the order of removals, without seeing whether the desk used slots or linked nodes.

Infer a rule from the next removal

Create two copies of the ticket pile. Remove one item, add D, then remove again under each policy. Record the removed sequence before looking at the results.

Operation	Stack	Queue
Remove	C	A
Add D	D goes on top	D goes to rear
Remove	D	B

Reading the result: The stack favours the newest remaining item. The queue favours the oldest remaining item. The distinction reappears in undo, request handling and the order in which a route explorer examines buildings.

Try it yourself: If there is exactly one waiting item, can one removal distinguish the policies?

Check your reasoning: No. Both must remove that item. A revealing test needs at least two distinguishable waiting items.

Another case: Three tickets with identical names

Two visitors both write “Sam” on a slip. The desk calls Sam twice. Can you use those calls alone to decide whether the desk used FIFO or LIFO? Design a stronger observation.

Feedback: Identical display names conceal which arrival left first. Attach arrival IDs S1 and S2 and record the returned IDs. FIFO removes S1 first, while LIFO removes S2 first. This separates a failure to observe identities from an actual failure of the service rule.

4.2 A stack of return trays

To add a tray to a stack, place it on top. To take a tray, remove the top one. Programmers call these operations push and pop. Peek or top reads the top item while leaving it in the stack. The behaviour is last in, first out, often abbreviated LIFO.

Write stack contents from bottom to top so the last written name is the top. This is a drawing convention; a real implementation need not grow upward. The convention matters because reversing it midway through a trace changes the apparent answer.

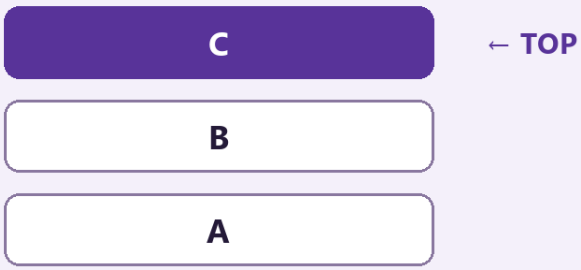


FIG-DS-04-02 The top exposed tray is C. Removing C reveals B. The oldest tray, A, remains at the bottom until all the later trays have been removed.

Look closer: A stack drawing can be horizontal. Mark the top, then push and pop at that same end. The familiar tray analogy is useful for direction, but the word top describes the permitted operation rather than gravity.

Distinguish looking from taking

Starting empty, push A, push B, peek, pop, push C and pop. Predict both returned values from the pops, and separately predict the final stack.

Step	Return if any	Stack bottom to top
Push A, push B	-	A B
Peek	B	A B
Pop	B	A
Push C	-	A C
Pop	C	A

Reading the result: Peek and pop can return the same value on successive operations because peek leaves that value available. Every pop requires a nonempty stack, or a documented empty-result policy. Underflow means an attempt to remove from an empty structure.

Try it yourself: Starting from the final state, how many further successful pops are possible?

Check your reasoning: One, returning A. The next pop encounters an empty stack and must follow the agreed empty behaviour.

Another case: Two pops after a peek

Push J, K and L. Pop, peek, pop, then peek again. Write the four observed return values and the final stack. State which two operations changed its length.

Feedback: The first pop returns L, leaving J and K. The next peek returns K without changing anything. The second pop returns K and the final peek returns J. The two pops reduce length from three to one; the two peeks leave it intact.

4.3 Undo the latest edit

Pip edits a shelf label from Oak to Elm, then from Elm to Ash. Pressing Undo once should restore Elm. Merely saving the final word Ash would not reveal the previous word. An undo system needs a history entry that can restore or reverse the most recent change.

For a tiny text field, push its previous value before replacing it. Undo pops a saved value and installs it as the current text. This design stores snapshots, which is easy to understand but may use space for large fields. Another design can store reversible edit instructions.

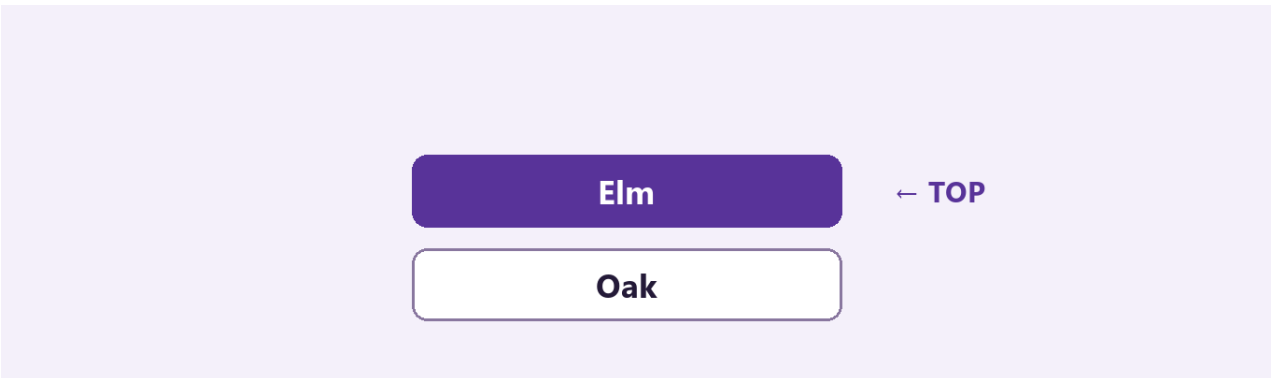


FIG-DS-04-03 Current text is Ash outside the stack. The top saved value is Elm, which is the text needed for the first undo. Keeping current text outside the history clarifies what a pop restores.

Look closer: Saving only the length of a title usually cannot support an undo of arbitrary text. “Oak” and “Elm” have the same length. Store information sufficient to recover the previous state the user expects.

Store enough history to reverse a change

Begin with Oak and an empty history. Make two edits and undo twice. Write the current text in a separate box so you do not accidentally push the replacement rather than the value being replaced.

Event	Current text	History bottom to top
Start	Oak	Empty
Replace with Elm	Elm	Oak
Replace with Ash	Ash	Oak, Elm
Undo once	Elm	Oak
Undo twice	Oak	Empty

Reading the result: A history of only the new values would require a different restoration convention and an explicit original state. Matching the stored information to the promised operation is an important part of data-structure design.

Try it yourself: What if the desk wants a third undo?

Check your reasoning: The history is empty, so report that there is no earlier edit. Inventing an earlier value would make undo unreliable.

Another case: Undo after a new edit

With current text Ash and history [Oak, Elm], undo once and then edit the current text to Fir. Predict the history immediately before the next undo. Why should restoring Elm still be possible?

Feedback: Undo restores Elm and leaves history [Oak]. Editing to Fir pushes the previous value Elm, producing [Oak, Elm] again. The next undo restores Elm. Any separate redo policy would need to decide what to do with the abandoned Ash branch.

4.4 Undo remembers a change, not just an item

A library editor changes a shelf from A1 to B4, then changes a title. Pressing Undo should reverse the title edit first. Saving only the name of the edited field is insufficient: the program also needs the old value. A stack organizes the order of reversal, while the content of each history record determines whether the reversal is possible.

For this simple editor, push a record containing the affected copy ID, field, old value and new value after each successful edit. Pop the latest record to undo it and restore its old value. Peek merely previews that record. A failed edit should not be recorded as though it succeeded. Real

editors may group several small actions into one undo step; this example deliberately treats one field change as one action.



FIG-DS-04-04 The most recent change is on top. The drawing represents history records, not physical books; popping one record triggers a reversal in another stored structure.

Investigate and explain

Start with copy 12 at shelf A1 and title Old. Change shelf to B4, change title to New, then undo once and twice. Record both the data and history. Explain why restoring the title cannot change the shelf in this trace.

Event	Shelf	Title	History depth
Initial	A1	Old	0
Shelf edit	B4	Old	1
Title edit	B4	New	2
First undo	B4	Old	1
Second undo	A1	Old	0

Reading the result: The history contains separate records for separate fields. The first pop restores only the title. The second restores the shelf. Undoing from an empty stack must produce an explicit no-action result. If the editor stores references to mutable records, it must ensure old values are actually preserved rather than accidentally reading the latest value twice.

Try it yourself: Can a stack implement redo by itself without saving undone changes?

Check your reasoning: No. Once a popped change is discarded, the information needed for redo may be gone. A second history stack can retain undone records; a new edit normally clears the obsolete redo branch under a stated policy.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Push A, push B, peek, pop, push C, pop, pop. Record every returned value and show the stack after every step. Then explain an undo application.

Worked guidance

Peek returns B without removal. The first pop returns B, the next returns C and the last returns A. The stack ends empty. Undo records must preserve the old state of the action being reversed, not only its name.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can trace push, pop and peek; design an undo record that preserves the information needed for reversal. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

DS-CH05 / CHAPTER 05

Queues

BY THE END

Trace FIFO operations, circular storage and explicit empty/full behavior.

A queue preserves waiting order. Physical position and completion order are separate ideas.

5.1 At the service window

Pip is handing out reserved books. Ada arrives before Bo, who arrives before Cy. Serving visitors in arrival order is first in, first out, or FIFO. Enqueue adds at the rear; dequeue removes from the front. A queue can also expose the front item without removing it.

A queue describes the order in which waiting jobs begin service. If several clerks work at different speeds, their completion order might differ. Keep the waiting line and the job already in a clerk's hands separate in your trace.

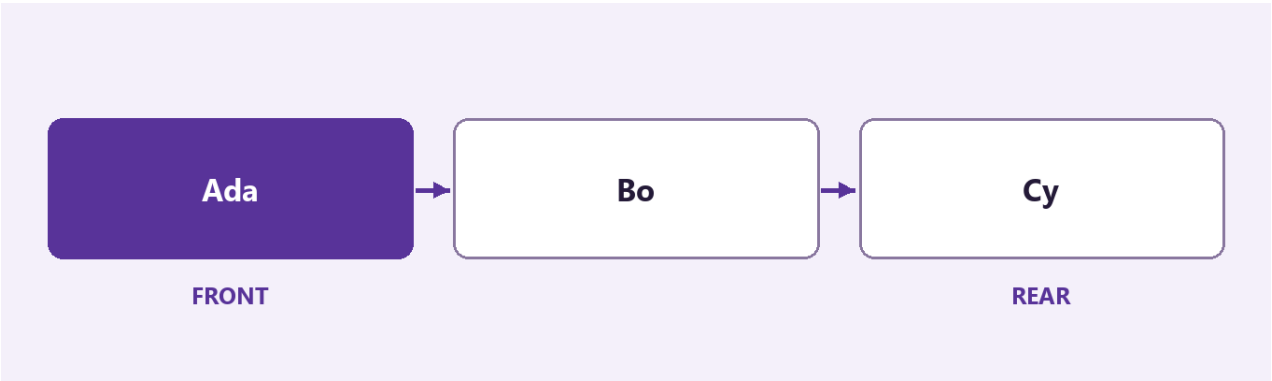


FIG-DS-05-01 Ada is at the front and Cy at the rear. New arrivals join after Cy. A dequeue removes Ada without moving her to the back.

Look closer: A visitor who leaves voluntarily may need to be removed from the middle of the waiting line. That request is outside the basic FIFO interface. A real service can add a cancellation policy without redefining normal dequeue.

Simulate an arrival during service

Begin with the pictured queue. Dequeue Ada, enqueue Di, dequeue Bo, then enqueue Eli. Draw an arrow where each new ticket enters. Predict the next person to be called.

Event	Removed	Waiting front to rear
Start	-	Ada Bo Cy
Dequeue	Ada	Bo Cy
Enqueue Di	-	Bo Cy Di
Dequeue	Bo	Cy Di
Enqueue Eli	-	Cy Di Eli

Reading the result: Cy is next even though Di and Eli arrived more recently. Giving a rush order priority changes the policy. It can be a legitimate design choice, but then the desk must explain which jobs are allowed to pass others.

Try it yourself: If the printer dequeues a job, does that prove it printed successfully?

Check your reasoning: No. Dequeue moves the job out of the waiting line. The system needs separate in-progress and completion or failure information.

Another case: A fast clerk and a slow clerk

Two clerks share a FIFO line. Ada starts first and needs five minutes, while Bo starts one minute later and needs one minute. Who finishes first? Does that result contradict FIFO?

Feedback: Bo finishes at minute two and Ada at minute five. FIFO governed which waiting job began next, not how long each took after leaving the line. Recording only the order of receipts would be insufficient to audit the order in which jobs started.

5.2 A queue wrapped around a rack

Shifting every ticket after each dequeue would be expensive in an array. Instead remember the index of the front ticket and the number of occupied slots. After a dequeue move the front one step. When you run off the end, wrap to index zero. This creates a circular queue in a fixed rack.

For capacity four and front index f, the next insertion uses (f + count) modulo 4 when the queue has room. Modulo gives the remainder after division. For example, 5 modulo 4 is 1. The count distinguishes an empty rack from a full rack.

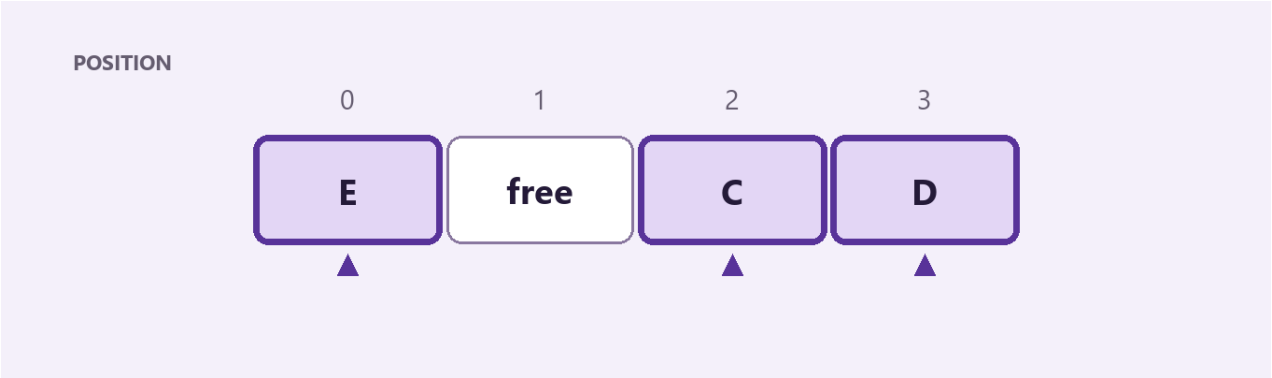


FIG-DS-05-02 Front is index 2 and count is three. Reading three slots with wraparound produces C, D, E. Physical left-to-right order would misread the waiting line.

Look closer: Distinguish the front position from the index of the most recent physical write. In the wrapped example E is written at index zero yet remains behind D. Reading slots numerically would promote an arrival unfairly.

Reuse a freed slot without shifting

Start with [A, B, C, D], front 0, count 4. Remove twice, add E, then add F. Record both front and count after each action. Predict where E and F are written.

Event	Front	Count	Logical queue
Remove A	1	3	B C D
Remove B	2	2	C D
Add E at 0	2	3	C D E
Add F at 1	2	4	C D E F

Reading the result: The spare slots at the beginning became the rear of the queue. No surviving item was copied. A full fixed rack must reject or defer another enqueue, or the implementation must explicitly grow it before writing.

Try it yourself: Why is writing at (front + count) modulo 4 unsafe when count equals four?

Check your reasoning: It points back to front. A write there would overwrite the oldest waiting ticket, so the capacity check comes first.

Another case: A full rack with a wrapped front

A circular rack of capacity four has front 3 and count four. In logical order its tickets are P, Q, R, S. Write the physical indices of all four tickets. Remove P and then enqueue T. Where is T stored?

Feedback: The indices are 3, 0, 1, 2. Removing P sets front to 0 and count to three. The insertion index is (0 + 3) modulo 4 = 3. T uses the newly freed slot, after Q, R and S in service order.

5.3 Circular storage, straight waiting order

Pip has four storage slots for waiting requests. Moving every remaining request left after each removal would work, but it does unnecessary work. Instead, remember where the logical front lies and let the rear wrap around to unused space. The waiting order stays straight even when the physical storage loops.

A circular queue can track front, the index of the next removal, and size, the number waiting. With capacity C, the next insertion position is (front + size) modulo C. Modulo means the remainder after division: 5 modulo 4 is 1. After removal, advance front to (front + 1) modulo C and decrease size. These rules require separate checks for empty and full; the same front position alone cannot distinguish those states.

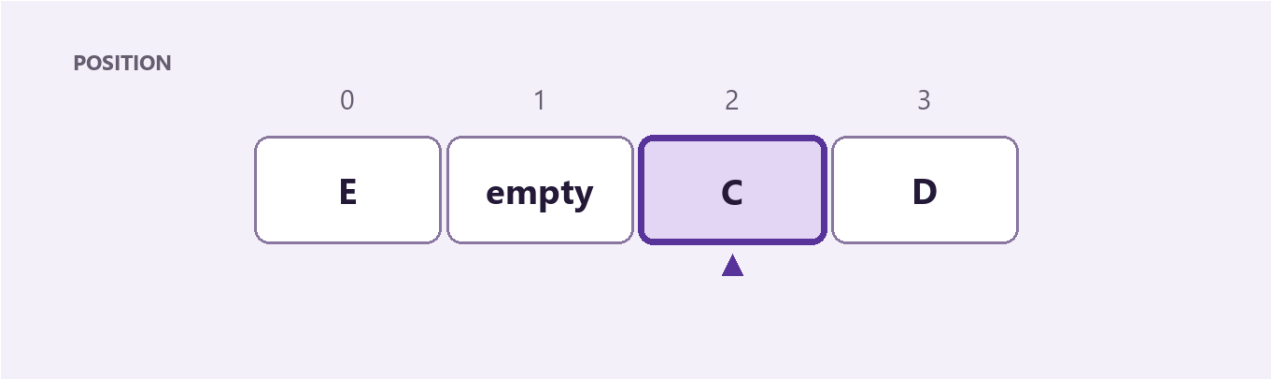


FIG-DS-05-03 Read the occupied slots from logical front 2, wrapping after index 3: C, D, E. The leftmost printed slot is not necessarily first in line.

Investigate and explain

Starting with front 2, size 3 and physical slots [E, empty, C, D], remove one request, then add F. Compute the index for each operation and reconstruct the final waiting order.

Step	Front	Size	Physical slots	Logical order
Start	2	3	E, empty, C, D	C, D, E
Remove C	3	2	E, empty, empty, D	D, E
Add F at 1	3	3	E, F, empty, D	D, E, F

Reading the result: The insertion index is (3 + 2) modulo 4 = 1. No waiting request moves. Clearing a removed physical slot is useful for showing the state; front and size establish which positions are logically occupied. A full queue needs an explicit policy, such as reject with a message or grow storage. Overwriting the oldest waiting request silently would break the service promise.

Try it yourself: Does FIFO guarantee the order in which two clerks finish?

Check your reasoning: No. It governs which waiting job is selected next. Different service times can make a later-started job finish earlier when several clerks work at once.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

With capacity four, enqueue A, B, C; dequeue twice; enqueue D, E. Use front and size to draw physical storage and logical order. State a full-queue policy.

Worked guidance

Starting front=0 gives occupied slots [E, empty, C, D], front=2, size=3 and logical order C, D, E. A full queue can reject the next request with a clear response or grow according to policy; it must not silently replace a waiting request.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can trace FIFO operations, circular storage and explicit empty/full behavior. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

DS-CH06 / CHAPTER 06

Trees and Hierarchies

BY THE END

Measure a rooted hierarchy, follow four traversals and justify a search-tree decision.

Parent links define hierarchy. Traversal order determines when a node is visited. Search-tree order can eliminate a subtree, but height determines how long the remaining path can be.

6.1 A folder with branches

Open the library folder. Inside are Art and Science, and inside Art are Ink and Clay. A flat list of these names hides which items belong together. A rooted tree gives every node except one a single parent, with one path down from the root to any node.

The root is the starting node. Children lie immediately below their parent. Siblings share a parent. A leaf has no children. A subtree consists of a node and everything descended from it. The position of a name on the printed page matters less than the lines connecting it.

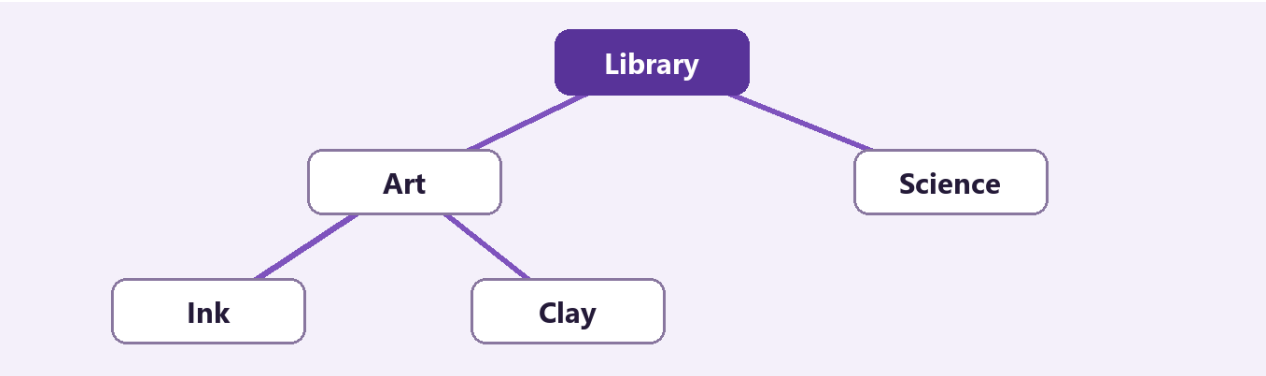


FIG-DS-06-01 Library has two children. Ink and Clay both belong to Art. Science has no children in this particular drawing, so it is a leaf even though its name sounds broad.

Look closer: Family relationships can be more complex than a rooted tree because a person can have two parents. A folder hierarchy with exactly one enclosing folder per item matches this introductory model more closely.

Name the relationships in a rooted tree

Sketch the tree, then cover the labels and reconstruct it from the parent records below. Predict the smallest subtree containing both Ink and Clay as descendants.

Node	Parent	Children
Library	None	Art, Science
Art	Library	Ink, Clay
Science	Library	None
Ink	Art	None
Clay	Art	None

Reading the result: The subtree rooted at Art contains Art, Ink and Clay. Calling Science a leaf describes its current position, not its subject. Adding a new child under Science would change its leaf status without changing its parent.

Try it yourself: If Ink were also a direct child of Science, would these links still form the stated rooted tree?

Check your reasoning: No. Ink would have two parents and two root-to-Ink paths. A more general graph can represent that relationship.

Another case: A folder moves home

Move Clay from beneath Art to beneath Science in the library tree. Identify one relationship that changes and one property that stays the same. Which node ceases to be a leaf?

Feedback: Science gains a child and ceases to be a leaf. Clay's parent changes, so Ink and Clay cease to be siblings. Clay is still two edges from Library. A single relinking can change several relationships without changing every depth.

6.2 Measure a tree in edges

Pip asks how many folder-opening steps lead from the library root to Ink. Count the links traversed. This count is the depth of Ink. The root has depth zero because it takes no move to get from the root to itself.

Height looks in the other direction. The height of a node is the greatest number of downward edges from that node to a leaf. Under our edge-count convention every leaf has height zero, and the height of the whole tree is the root's height. Some books count nodes, so record the convention when comparing answers.

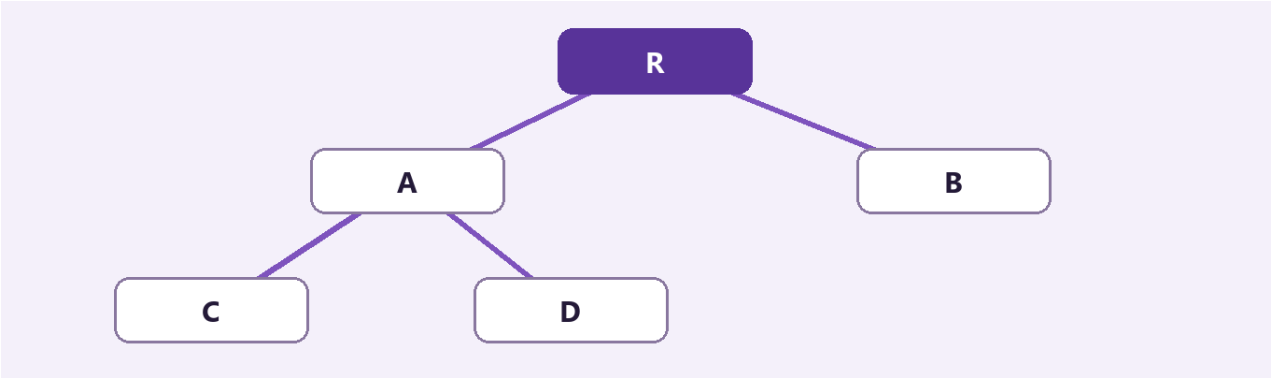


FIG-DS-06-02 C and D have depth two. B and A both have depth one, but A has height one while B has height zero. Nodes on the same level need not have equal heights.

Look closer: Depth from the root cannot be inferred from the number of children. A leaf may appear immediately below the root or at the end of a long branch. Count the links in the actual path.

Distinguish depth from height

List the edge paths from R to every leaf. Then cover the depth and height columns and calculate them separately. Use the longest downward path when a node has several children.

Node	Depth	Height
R	0	2
A	1	1
B	1	0
C	2	0
D	2	0

Reading the result: For a single-node tree the root is a leaf. Its depth and height are both zero. The concept measures relationships, so a count of zero is meaningful. A root with one child has height one regardless of how the two labels are spelled.

Try it yourself: Attach E below C. Which heights change?

Check your reasoning: C rises to one, A to two and R to three. Their depths stay fixed. E has depth three and height zero. B and D are unaffected.

Another case: Height after removing leaves

From the R, A, B, C, D tree, remove nodes C and D. Compute the new heights of A and R, then identify which remaining depths changed.

Feedback: A becomes a leaf of height zero. R now has two leaf children and height one. R stays at depth zero, while A and B remain at depth one. Height depends on descendants, whereas a surviving node's depth depends on its path from the root.

6.3 A binary shape and a search rule

A binary tree permits at most two children per node, distinguished as left and right. A node may have just a right child. A binary search tree adds an ordering rule. With distinct numeric keys, every key in the left subtree is smaller than the node's key, and every key in the right subtree is larger.

The word “subtree” does serious work. Checking only one parent-child edge at a time can miss a violation inherited from an ancestor. The ordering rule allows an entire branch to be eliminated when searching, much as sorted order allowed a region of an array to be eliminated.

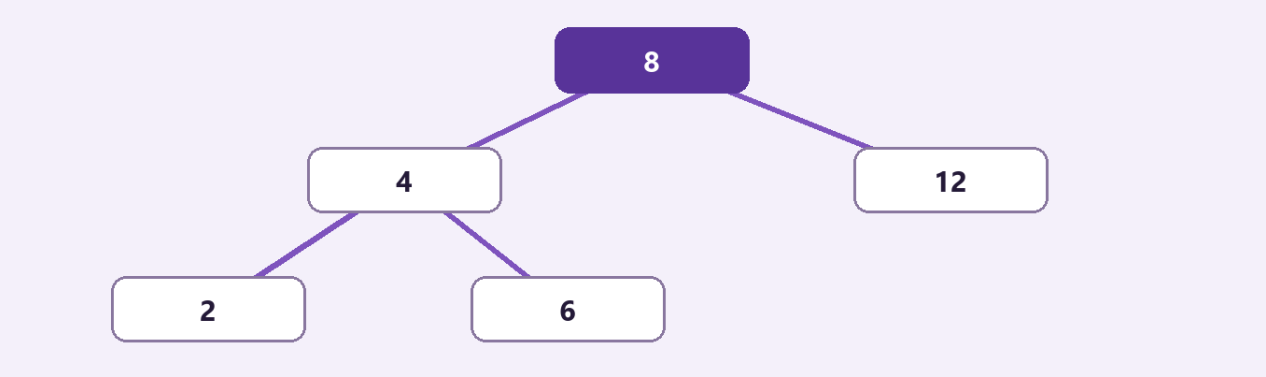


FIG-DS-06-03 All keys under 4 lie below 8, and 2 < 4 < 6 within that subtree. The right child 12 lies above 8. Each line connects a parent to one child.

Look closer: A binary search tree can validly have no right subtree. The ordering rule describes keys wherever descendants exist. It does not require both subtrees to contain a matching number of keys.

Test the entire subtree

Replace the 6 in the picture with 10 while leaving all edges in place. Test the ordinary binary shape, the local relationship to 4, and the constraint inherited from 8.

Property	Original 6	Replacement 10
At most two children	Pass	Pass
Right of 4 must exceed 4	Pass	Pass
Left subtree of 8 must be below 8	Pass	Fail

Reading the result: The replacement still makes a binary tree, but ordinary search for 10 would turn right at 8 and miss it. The order property is what justifies using a single branch. Without that property you may have to search both.

Try it yourself: How could a search tree handle repeated keys?

Check your reasoning: Declare a consistent policy, such as storing a count with each distinct key. The distinct-key examples here avoid requiring a duplicate policy.

Another case: A constraint inherited twice

In a search tree rooted at 20, the left child is 10 and its right child is 15. A learner attaches 22 as the right child of 15. Test the new placement against all relevant ancestor constraints.

Feedback: The new key exceeds 15 and 10, which fits those local and intermediate right turns. But it lies inside the left subtree of 20, where every key must be below 20. Search for 22 would go right from the root and miss the inserted node.

6.4 Three walks past the same nodes

To traverse a tree is to visit all its nodes in a stated order. At a node with two children, imagine three moments: arriving, returning from the left subtree, and returning from the right subtree. Record the node at a different moment and the traversal order changes.

Preorder records the node before its children. Inorder records left subtree, node, right subtree. Postorder records both subtrees before the node. Always finish a subtree by the same rule before resuming its parent. An empty child contributes no visit.

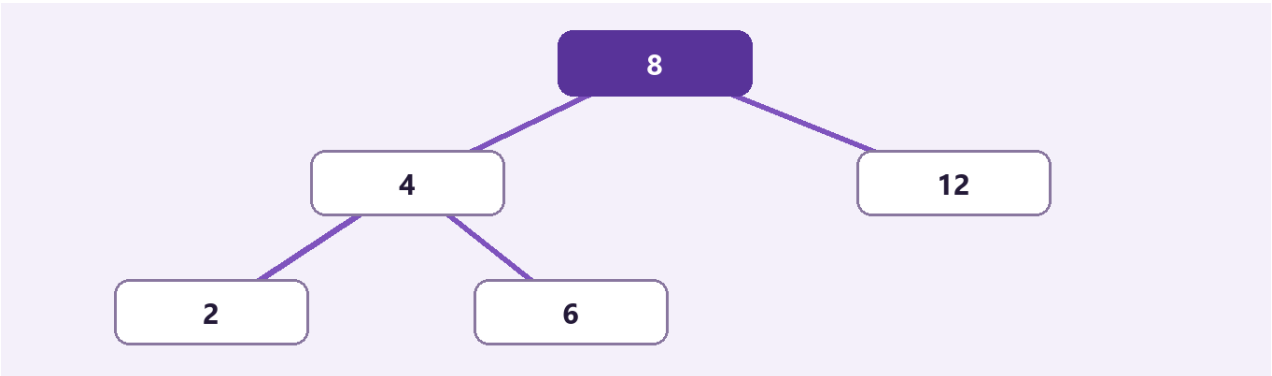


FIG-DS-06-04 For the left subtree rooted at 4, the three orders are 4 2 6, 2 4 6, and 2 6 4. Insert that whole sequence into the corresponding walk from 8.

Look closer: Write the three orders for a one-node tree. They all give the same single visit. A branching example is needed to distinguish the rules, just as multi-item cases distinguish a stack from a queue.

Put the visit at a different moment

Trace the tree with a pencil. Write down a node only at the moment specified by the selected rule. Draw a small return arrow to the parent when a subtree is finished.

Traversal	Left subtree sequence	Whole tree
Preorder	4 2 6	8 4 2 6 12
Inorder	2 4 6	2 4 6 8 12
Postorder	2 6 4	2 6 4 12 8

Reading the result: Inorder happens to produce ascending keys because this is a binary search tree. On a general binary tree it still gives a defined traversal but need not sort anything. Postorder suits a task where children must be processed before their parent.

Try it yourself: Which traversal could create a parent folder before creating its children?

Check your reasoning: Preorder visits the parent first. Postorder would suit removing a complete folder tree when children must be removed first.

Another case: One right child only

Draw root 4 with an empty left child and a single right child 6. Write preorder, inorder and postorder. Which two orders coincide, and why is the missing child important?

Feedback: Preorder and inorder both give 4, 6. The inorder walk visits the empty left subtree before recording 4, adding nothing to the sequence. Postorder gives 6, 4. Identical traversal sequences can occur for particular shapes even though the rules differ.

6.5 A level at a time

Pip wants a directory listing grouped by distance from the root. It should list both children of the root before grandchildren. Level-order traversal achieves this with the queue rule you already know.

Enqueue the root. While the queue is nonempty, dequeue the front, record that node, then enqueue its children from left to right. Older waiting nodes leave before newly discovered grandchildren. The queue is a frontier, the set of discovered nodes that still await their turns.

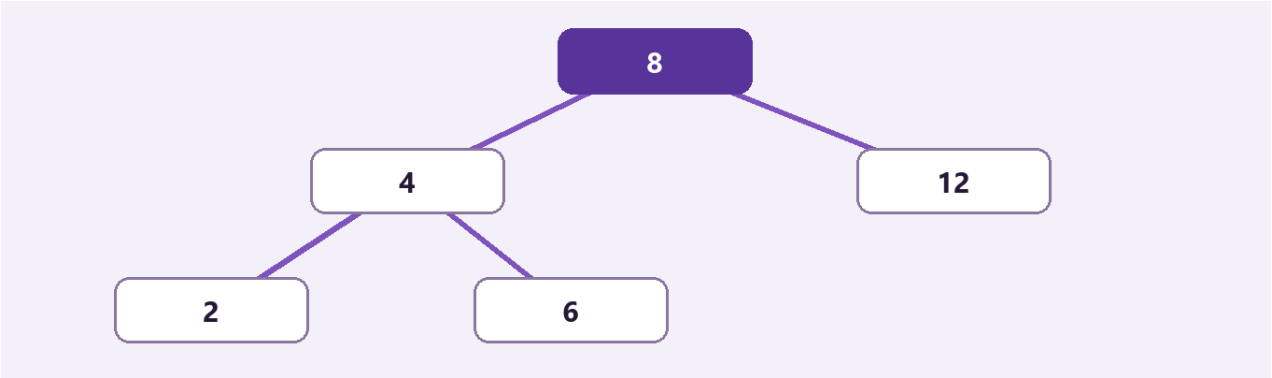


FIG-DS-06-05 Reading across each depth gives 8, then 4 and 12, then 2 and 6. The queue remembers 12 while the children of 4 are discovered.

Look closer: The queue can temporarily hold many nodes even when the tree has short root-to-leaf paths. Draw a wide level and count waiting slips. Depth and the number of pending visits describe different resource demands.

Use a queue to remember the frontier

Use one slip per node and work through the queue states. Predict the front just after 4 has been processed. Keep the visit list separate from the waiting queue.

Removed	Children enqueued	Queue after
8	4, 12	4 12
4	2, 6	12 2 6
12	None	2 6
2	None	6
6	None	Empty

Reading the result: The visit order is 8, 4, 12, 2, 6. At the second step the queue contains nodes from two depths. It still processes them in the correct order because enqueue adds to the rear.

Try it yourself: Would a stack produce the same guarantee about levels?

Check your reasoning: No. A newly added grandchild could leave before a previously waiting child. The choice of frontier determines the exploration order.

Another case: Reverse the order within a level

Run level order on the five-key tree but enqueue right children before left children. Does this destroy the level-by-level guarantee? Predict the full resulting sequence.

Feedback: The result is 8, 12, 4, 6, 2. Nodes still leave by increasing depth because the queue keeps all earlier discoveries ahead of later ones. Changing sibling order changes the order within each level, not the separation between levels.

6.6 The cost of a tall tree

A search begins at the root. On equality it returns success. For a smaller target follow the left child; for a larger target follow the right child. A missing child where the target would have to lie proves absence. The number of inspected keys depends on the height.

To insert a distinct key, follow the same search until the required child is empty, then attach a new leaf there. Inserting keys in ascending order can produce a long one-sided chain. Binary describes the allowed number of children, not a guarantee that both sides have similar heights.

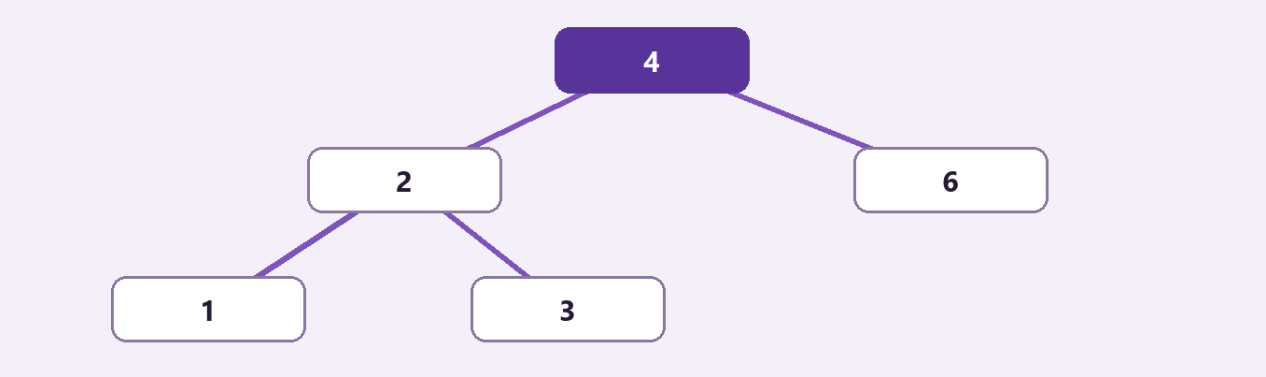


FIG-DS-06-06 Here the first key 4 separates smaller keys from 6. A second construction can place the same five keys in a chain by inserting them in ascending order.

Look closer: If a path search returns missing, identify the exact empty child link that ended it. This small detail shows how a global fact about sorted subtrees supports a local stopping decision.

Grow the same keys two ways

Use slips 1, 2, 3, 4, 6. Build one tree by inserting 4, 2, 6, 1, 3 and another by inserting 1, 2, 3, 4, 6. Predict the comparisons for target 6 and for missing target 5. Record the visited keys in each tree.

Target	First tree	Ascending-insert tree
6	4, 6	1, 2, 3, 4, 6
5	4, 6, missing left	1, 2, 3, 4, 6, missing left

Reading the result: Both trees satisfy exactly the same ordering rule and represent exactly the same set. Their shapes produce very different search costs. The longest path could contain every key; a tree with short paths can make search much cheaper.

Try it yourself: Why would a campus walkway map be awkward as a rooted tree?

Check your reasoning: Two different paths can lead to the same building and a walkway can bring Pip back to the start. Volume II, Chapter 7 uses graphs to retain those possibilities.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Insert keys 20, 10, 30, 15 into a binary search tree. Draw parents and children, compute edge-count height and give inorder traversal. Compare the height after sorted insertion 10, 15, 20, 30 into an initially empty ordinary BST.

Worked guidance

The first tree has root 20, left 10, right 30 and 15 as the right child of 10; height 2. Inorder is 10, 15, 20, 30. Sorted insertion forms a right chain of height 3. Shape affects search cost even with the same keys.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can measure a rooted hierarchy, follow four traversals and justify a search-tree decision. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

DS-CH07 / CHAPTER 07

Graphs and Route Models

BY THE END

Represent a campus, trace breadth-first and depth-first exploration, and distinguish fewest edges from least cost.

A graph preserves relationships that a tree can lose, including alternate routes. Queue exploration minimizes unweighted hop count. To compare travel time, use the edge weights.

7.1 Turn a campus into a graph

A student can walk from the Library to the Lab directly or via the Café. A tree would have to throw away one route or draw the Lab twice. A graph represents objects as vertices, also called nodes, and relationships between pairs as edges.

For this campus let L mean Library, C Café, B Lab, and G Gate. A line means there is a walkway. A number on it gives walking minutes. Lines without arrowheads can be used in either direction in this example. The drawing positions help you see the connections; the figures on the edges supply the actual costs.

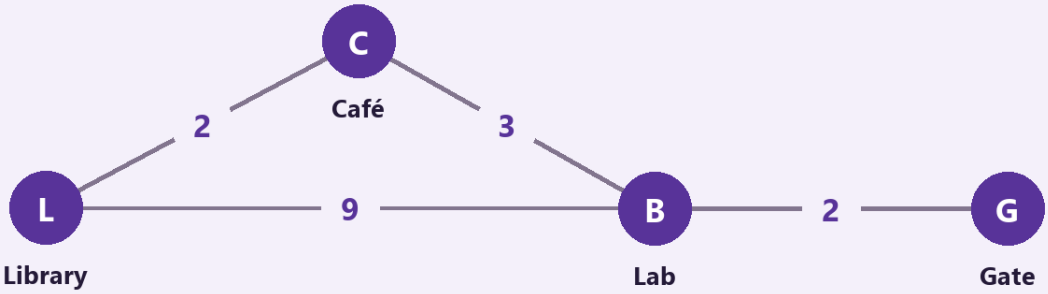


FIG-DS-07-01 A direct L-B walk takes 9 minutes. L-C takes 2 and C-B takes 3. B-G takes 2. These four edges are the entire fictional map.

Look closer: Decide what a vertex represents before drawing a graph. Modelling rooms, buildings or whole campuses produces different graphs for the same physical world. Each representation can answer a different planning question.

Decide which details become connections

Draw the map from its edge records, then list every single-edge move from L. Distinguish the neighbouring vertices from vertices reachable only by making additional moves.

Walkway	Minutes	Endpoints
Library to Café	2	L and C
Café to Lab	3	C and B
Library to Lab	9	L and B
Lab to Gate	2	B and G

Reading the result: The neighbours of L are B and C. G is reachable but is not an immediate neighbour. A path is a sequence of adjacent vertices. A cycle can lead back to its start, such as L, C, B, L. This graph is connected because every vertex can reach every other.

Try it yourself: If G's only walkway closes, what changes?

Check your reasoning: G remains a building and therefore a vertex. With B-G removed it is isolated, and the map is disconnected. Keeping vertices separately prevents an isolated building from disappearing.

Another case: A map with an extra bridge

Add a new two-way C-G walkway to the original campus. Which vertices gain a neighbour? Does the direct edge change the fact that L and G are not immediate neighbours?

Feedback: Only C and G gain each other as neighbours. L still has no direct L-G edge, although there is now an additional two-leg route via C. Reachability, adjacency and route count express different properties of the same modified drawing.

7.2 A one-way passage changes the question

A campus shuttle may travel one way around a loop. A directed edge has a source and a destination. In a directed graph, reachability from one building to another need not imply a route back. An undirected walkway can be represented by two opposing directed edges when needed.

Weight and direction express different information. A directed edge can have a weight or have none. An unweighted graph can answer whether a route exists or how many edges it uses. To ask about travel time, attach times and specify how times combine along a route.

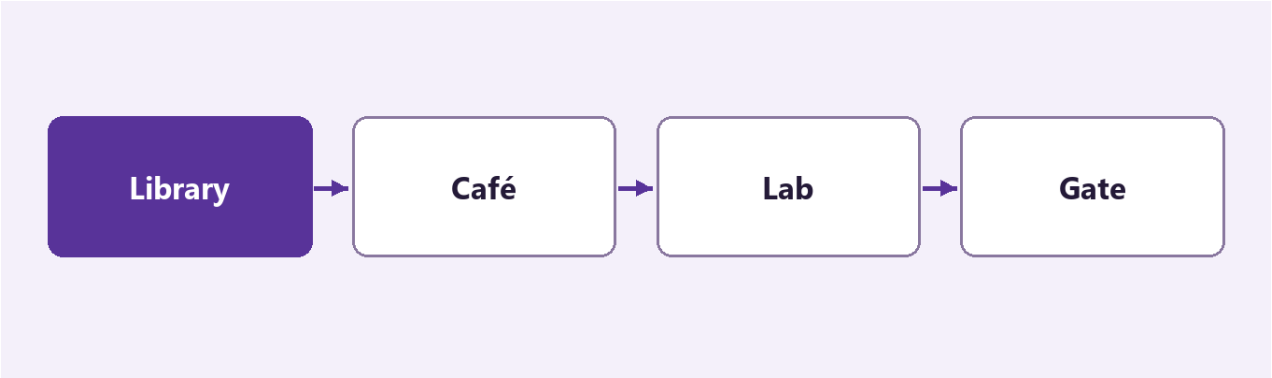


FIG-DS-07-02 For this separate one-way shuttle, arrows give the only permitted moves. The Gate has no outgoing arrow. This diagram replaces the walking map for this exercise.

Look closer: Direction and cost can vary independently. A hill may permit both directions but take longer uphill. Two directed edges with separate weights can preserve those unequal travel times.

Test reachability in both directions

A student boards at Café. List the possible destinations using zero, one and two shuttle legs. Repeat from Gate. Decide whether an instruction to “use the nearest stop” gives enough information to design a return journey.

Start	One directed leg	Two directed legs
Library	Café	Lab
Café	Lab	Gate
Lab	Gate	None
Gate	None	None

Reading the result: A zero-leg trip stays at its starting point. From Café, Gate is reachable in two legs, while Library is unreachable. A route planner that treats the arrows as ordinary lines will offer illegal journeys.

Try it yourself: If a new Gate -> Library shuttle is added, which structural feature appears?

Check your reasoning: The four arrows form a directed cycle. Each stop can now reach every other by following the loop, although the return trip may use different edges.

Another case: One-time cost or two-way cost?

A two-way walkway takes two minutes downhill and four minutes uphill. Show how to represent it without losing direction-dependent cost. Can a single undirected weight encode both numbers under the book's convention?

Feedback: Use two directed edges with opposite endpoints and different weights. One edge gives the downhill time and the reverse gives the uphill time. A single undirected weight would assign the same cost in both directions, so it cannot answer both queries correctly here.

7.3 Two ways to store the same map

A drawing is excellent for people, but Pip needs a repeatable way to find neighbours. An adjacency list gives each vertex a list of its neighbours, optionally with weights. An adjacency matrix uses a grid with one row per starting vertex and one column per possible destination.

In an unweighted matrix, use 1 for an edge and 0 for no edge. In a weighted matrix choose a distinct no-edge marker if zero could be a valid weight. An undirected graph produces matching entries on opposite sides of the diagonal. A directed graph need not.

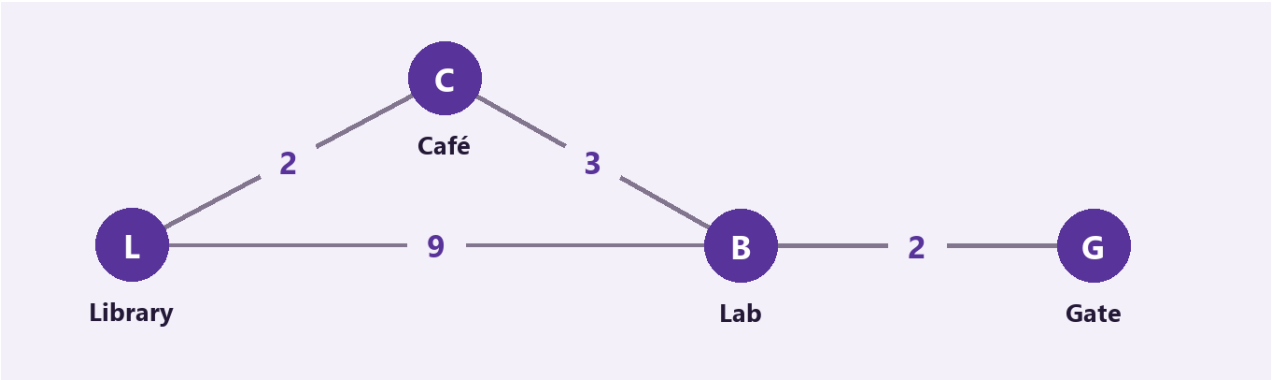


FIG-DS-07-03 Return to the two-way walking map. Four vertices require sixteen possible matrix entries. Each real walkway appears in two neighbour lists, once from each endpoint.

Look closer: A matrix entry of zero can mean no edge in the unweighted convention. On a map that includes zero-minute transfers, using the same zero for absence would lose information unless a separate presence marker is kept.

Translate between a list and a matrix

Use the order L, C, B, G for both axes. Fill an unweighted matrix, then write the weighted list for B. Verify your entries by reading both representations back into a drawing.

From to	L	C	B	G
L	0	1	1	0
C	1	0	1	0
B	1	1	0	1
G	0	0	1	0

Reading the result: B's list is L(9), C(3), G(2). The unweighted grid loses those times while preserving connectivity. A matrix makes a known pair easy to check. Lists avoid reserving an entry for every absent edge and make it convenient to enumerate actual neighbours.

Try it yourself: How many marked cells should this undirected grid contain?

Check your reasoning: Eight. Each of the four edges is recorded in both directions. This count assumes no self-loops and no repeated parallel walkways.

Another case: One closure, two edits

The two-way C-B walkway closes. Change the unweighted matrix on this page without altering any building labels. Which entries must become zero and why is one edit insufficient?

Feedback: Change row C column B and row B column C from one to zero. Editing only the first would leave the second direction usable in the stored map. That would represent a one-way closure, a different event from closing the two-way walkway.

7.4 Breadth first with a queue

To explore the area around the Library, Pip first considers buildings one edge away, then two edges away, and so on. Breadth-first search, abbreviated BFS, uses a queue. Mark a vertex discovered as soon as it joins the queue, so two different neighbours cannot add it twice.

When a vertex is first discovered, record the vertex that discovered it as its predecessor. Following these predecessors backward from a destination reconstructs a route with the fewest edges on an unweighted graph. The queue's depth order supplies this guarantee.

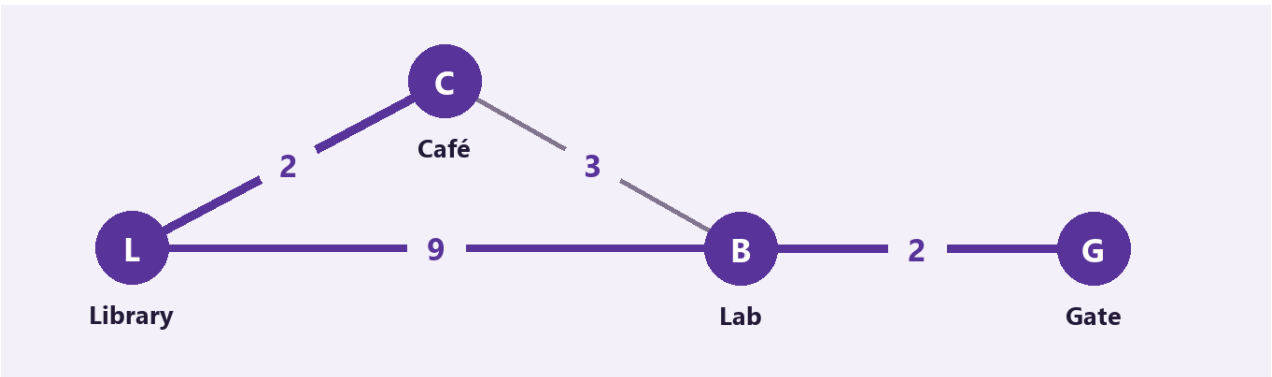


FIG-DS-07-04 Start at L and consider its neighbours B before C. Both are one edge from L. G is discovered from B. The time labels play no role in this search.

Look closer: The first predecessor of a vertex depends on the chosen neighbour order when two shortest routes tie. The route returned may change, while the number of edges in a shortest route stays the same.

Find a route with fewest edges

Start with L discovered and queued. For each removed vertex, add only previously undiscovered neighbours in alphabetical letter order. Write the predecessor when a vertex first joins, and keep it on later encounters.

Removed	New discoveries	Queue after
L	B from L, C from L	B C
B	G from B	C G
C	None	G
G	None	Empty

Reading the result: Following predecessors from G gives G, B, L. Reverse that order to obtain L, B, G with two edges. Its total time is 9 + 2 = 11 minutes. The route has a minimum number of legs, which is a different objective from minimum walking time.

Try it yourself: Why mark on enqueue?

Check your reasoning: Without early marking, two vertices can both add the same undiscovered neighbour before its first removal. Early marking gives it one discovery record and one place in the queue.

Another case: Two discoveries at the same depth

Add a C-G edge to the unweighted map. Explore from L with C before B and record G's predecessor. Now reverse the neighbour order. What changes about the route, and what stays guaranteed?

Feedback: With C first, G is first discovered from C. With B first, G is first discovered from B. Both routes have two edges and therefore satisfy the BFS minimum-edge guarantee. Neighbour order breaks a tie rather than invalidating the search.

7.5 Depth first down a branch

Another explorer follows one available path as far as possible, then returns to the most recent junction with an unexplored choice. This is depth-first search, or DFS. The most recent unfinished junction belongs on top of a stack.

Use this convention: at the top vertex, choose its alphabetically first unvisited neighbour, mark it, and push it. If there is none, pop the top and backtrack. A visited set prevents a cycle from sending the explorer around forever. Other implementations may order neighbours differently.

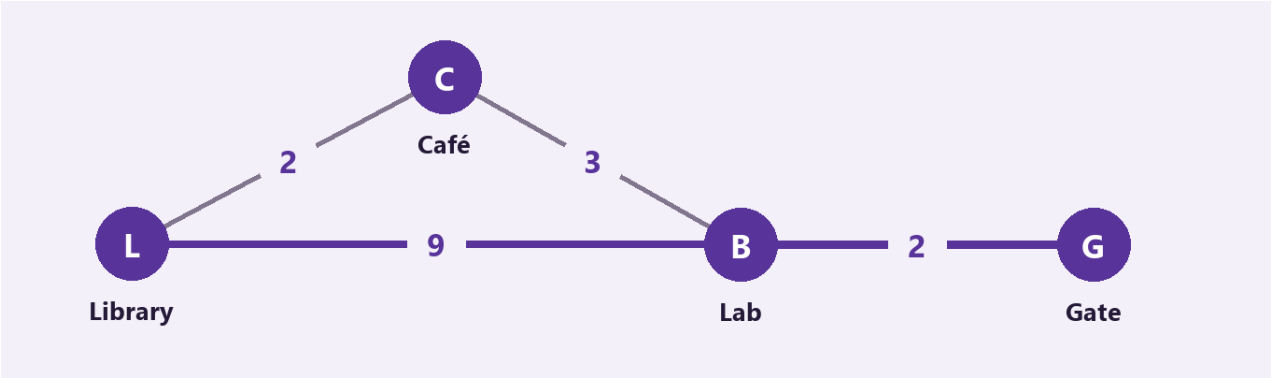


FIG-DS-07-05 With B before C at L, a depth-first walk reaches B, then C, before backing up to B to explore G. Keep the stack separate from the permanent visited list.

Look closer: A backtrack in DFS is a change in the exploration procedure. On a directed graph it need not be an allowed physical return journey. The stack stores the unfinished search, not a shuttle ticket.

See why neighbour order matters

Start at L with stack [L]. At each move record the current route stack. Predict which vertex is discovered third and which action first shortens the stack.

Action	Stack bottom to top	Discovered so far
Start	L	L
Push B	L B	L B
Push C	L B C	L B C
Pop C	L B	L B C
Push G	L B G	L B C G

Reading the result: All remaining moves are pops because every reachable vertex has been discovered. Popping C erases it from the unfinished route, but C remains visited. Losing that distinction would allow Pip to rediscover C each time it returns to B.

Try it yourself: Does the first DFS route to a goal always minimize the number of edges?

Check your reasoning: No. If C is chosen first at L, DFS can reach B through C even though a direct L-B edge exists. DFS guarantees exploration under these rules, not shortest routes.

Another case: A visited vertex still has unfinished work

During the illustrated DFS, after C is popped the stack is [L, B]. Which vertex was visited earlier but still needs attention? What would happen if Pip discarded every vertex immediately after marking it visited?

Feedback: B is already marked but has the unvisited neighbour G. Its stack entry preserves where the exploration must resume. A visited marker prevents rediscovery; it does not by itself say that every outgoing edge has been considered.

7.6 Which route should the robot take?

“Shortest” sounds unambiguous until two routes disagree. One has fewer turns, another takes fewer minutes. Define the route cost first. For the walking map add the minutes on every traversed edge. The number of vertices visited is a separate count.

All our edge weights are positive. A route containing a cycle can be shortened in time by removing the cycle and continuing from its starting vertex. On this tiny graph it is enough to compare the simple routes, which never repeat a vertex. Larger maps need more systematic methods.

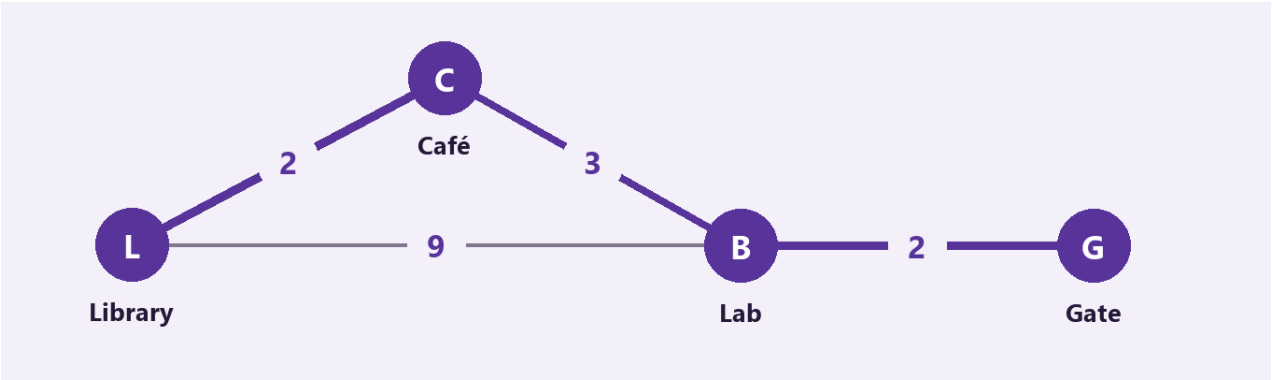


FIG-DS-07-06 Every route to G must use B-G at the end. The meaningful choice is how to reach B from L: directly or through C.

Look closer: The sum of edge weights assumes independent leg costs. Waiting for a bus connection may depend on arrival time. A richer route problem would need a timetable or additional state.

Compare cost, legality and evidence

Use cards for the four buildings and draw the four walkways. Predict the fewest-edge route from L to G and the least-time route. Add the edge weights yourself. Then close C-B and reconsider both answers. Record the reason for each change.

Route	Edges	Time
L-B-G	2	9 + 2 = 11
L-C-B-G	3	2 + 3 + 2 = 7
After C-B closes, L-B-G	2	11

Reading the result: The three-edge route wins on time by four minutes. After the closure it becomes illegal, however attractive its old cost looks. A route algorithm must use the current map and a clearly stated objective.

Try it yourself: Could the map tell us which book a visitor would enjoy?

Check your reasoning: It tells us how to reach a destination once chosen. Preferences require different information. Volume III, Chapters 1 to 4 asks how numerical examples can support a prediction about an unfamiliar case.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Draw two-way edges LC 2, CB 3, LB 9, BG 2. Compare L-to-G fewest-edge and least-time routes. Close CB and then BG, revising the result after each change.

Worked guidance

L-B-G has two edges and takes 11; L-C-B-G has three and takes 7. Closing CB leaves the 11-minute simple route. Closing BG makes G unreachable. BFS minimizes hops in the unweighted objective; it does not minimize arbitrary travel weights.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can represent a campus, trace breadth-first and depth-first exploration, and distinguish fewest edges from least cost. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

DS-CH08 / CHAPTER 08

Choosing and Combining Data Structures

BY THE END

Choose representations using workload, preserve identity across views and test component boundaries.

The right question is which operations and promises the application needs. Most useful systems combine several structures.



ART-DS-CH08 One library service combines records, request order, routes and optional suggestions. The technical blueprint will specify their exact responsibilities.

8.1 Build a decision from a workload

A workload describes the operations a system performs and how frequently it performs them. A catalogue read thousands of times between updates has different priorities from a queue that changes at every arrival. Begin with a small representative day rather than choosing a structure because it is familiar.

Write the required behavior first, including order, identity and error responses. Next list common and expensive operations. Finally compare possible representations under those assumptions. A sorted array can support binary search by its sorting key, but an insertion may shift a suffix. A singly linked list can splice after a known node, but locating an arbitrary position requires

traversal. A balanced search tree is an optional extension for ordered updates; an ordinary binary search tree can become a long chain.

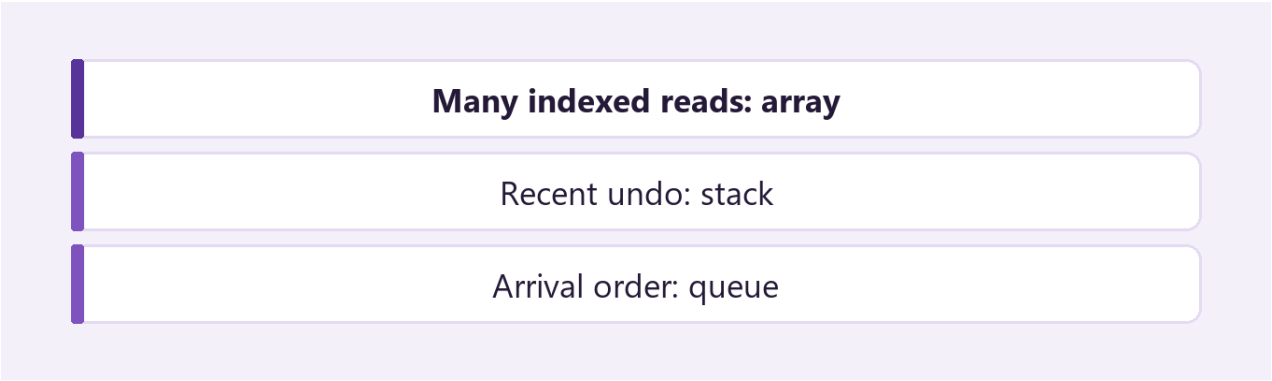


FIG-DS-08-01 These choices answer different requirements. The panels are starting points for reasoning, not an instruction to force an entire application into one container.

Investigate and explain

Compare two catalogue designs for 100 lookups and two insertions. Use the supplied simplified costs: scan 100 per lookup; sorted lookup 7 per lookup; sorted insertion 100 each; unsorted append 1 each. Do not add a preparation cost because both initial catalogues are already prepared.

Design	Lookup budget	Update budget	Total
Unsorted array	100 x 100 = 10000	2 x 1 = 2	10002
Sorted array	100 x 7 = 700	2 x 100 = 200	900

Reading the result: Under this cost model the sorted array has the smaller total budget. This is an estimate in declared work units, not a measured time. The outcome can change if there are many more updates, first-position scan hits, or expensive key comparisons. The calculation is useful because another designer can inspect and revise each assumption instead of arguing from a slogan.

Try it yourself: Does binary search help if the row is sorted by title but queried by numeric copy ID?

Check your reasoning: No. The comparison key must match the maintained order. Use a scan by ID or maintain an additional ID-sorted index.

8.2 Several views of the same record

The library wants to list books by title while locating copies by ID. Duplicating the entire catalogue into unrelated copies seems convenient until one copy says available and the other says on loan. Multiple views are useful only when the relationships between them remain consistent.

An index is an auxiliary arrangement that helps find an underlying record. For example, an ID-sorted list of pairs can map copy IDs to record references while a title-sorted view supports browsing. The record remains the source of current availability. Updating one attribute need not change every ordering, but changing an indexed key requires repairing that index. These are design ideas; implementing concurrent database transactions is beyond this introductory chapter.



FIG-DS-08-02 An arrow means refers to, not copies every field. The final check reads the authoritative current record rather than a remembered display value.

Investigate and explain

Copy 24 appears in an ID index and a title view. Its availability changes to held for R1. Explain which record needs changing and what a later request must check. Then suppose the title changes from River Walks to Alpine Rivers.

Change	Required repair	Unchanged fact
Availability becomes held	Update canonical availability	Copy ID stays 24
Title renamed	Update record and title ordering	ID ordering stays valid
Record removed	Remove stale references from views	Historic receipts retain their own evidence

Reading the result: A view that caches availability also needs a refresh policy. Otherwise a correct lookup can lead to an outdated decision. This is why the capstone separates finding a record from checking whether the requested copy may be allocated. Correct storage design includes update rules, not just an attractive initial diagram.

Try it yourself: Can the title view safely store only array positions forever?

Check your reasoning: Only if those positions remain stable or are repaired after moves. A lasting ID or stable record reference avoids confusing a shifted position with an unchanged identity.

8.3 Test the promises between components

The request queue can be correct, the catalogue can be correct, and a delivery can still fail when the two parts disagree about a status. A component boundary is the place where one part hands information to another. Name exactly what crosses it and what a failure means.

Use request IDs to connect the queue, hold record and receipt. A status such as waiting, held, delivered or failed describes a state, and an allowed transition describes a valid change. Delivery requires a confirmed handoff; finding a route is only preparation. A system test follows a whole request across these boundaries. A unit test isolates one operation. Both kinds of evidence matter because local correctness alone does not establish that the pieces cooperate.



FIG-DS-08-03 This is the success path. A missing copy or unreachable destination needs an explicit failure branch and an appropriate release policy.

Investigate and explain

Two requests R1 and R2 ask for the same initially available copy. One clerk serves them in order. Write the first state change that prevents both requests from claiming the copy. Then design an expected result when the only route closes before departure.

Event	Copy state	Request result
R1 accepted for service	Held for R1	R1 preparing
R2 checked while hold remains	Held for R1	R2 unavailable
R1 route becomes impossible before departure	Available after release	R1 failed, human informed

Reading the result: A hold associates a copy with one specific request. A route failure must not produce a delivered receipt. This trace assumes one clerk and a parcel still at the library; concurrent clerks or failures after departure require additional coordination and recovery policies. Stating the limit is part of a useful design, because it prevents a small classroom model from being mistaken for a production system.

Try it yourself: What if the queue stores only copy IDs and two people request the same copy?

Check your reasoning: The entries do not uniquely identify the requests. Add separate request IDs and link each to its copy, destination and status.

8.4 Choose a policy for a real desk

A student help desk has two jobs. It must serve waiting questions in arrival order and let a clerk undo recent changes to a response. One storage rule cannot express both promises with the same removal operation. Give each job its own structure.

Separating the structures clarifies what each record means. A queue item might hold a request identifier and question. A stack entry might hold the previous wording of an edit. The next item to process depends on which operation the clerk chooses.



FIG-DS-08-04 A single service window can use both rules at once. Trace the stack and the queue independently, then note the event at which their contents change together.

Look closer: A stack used for undo should identify the document or job it belongs to. If two visitors' histories share an undifferentiated stack, the latest edit globally may belong to the wrong visitor.

Run two disciplines on identical arrivals

Use three question slips and two edit slips. Enqueue Q1 and Q2, start Q1, save edits E1 then E2, enqueue Q3, then undo one edit. Predict the next question and which edit is undone. Record the state after each event.

Event	Waiting queue	History
Admit Q1, Q2	Q1 Q2	Empty
Start Q1	Q2	Empty
Save E1 then E2	Q2	E1 E2
Admit Q3	Q2 Q3	E1 E2
Undo last edit	Q2 Q3	E1

Reading the result: Q2 remains next throughout the edits. Undo removes E2. A bug that cancels Q3 when Undo is pressed confuses two histories with different meanings. Draw the information boundary before deciding on a structure.

Try it yourself: What structure fits one folder that contains shelves, each of which contains books?

Check your reasoning: Parent and child relationships form a hierarchy. A tree makes that hierarchy explicit, which lets us ask questions beyond which item leaves next.

Chapter lab: show what you can do

Prepare: Use paper, a pencil and the definitions in this chapter. Plan 15-25 minutes. Complete a first attempt before reading the worked guidance. Keep the trace or diagram as evidence.

Design a library blueprint with a catalogue, request queue, undo history and route map. Name each structure, what it stores and its required invariant. Trace one missing-book request through the boundaries.

Worked guidance

A catalogue stores copy records; a queue stores request IDs in arrival order; an undo stack stores reversible changes; a weighted graph stores current legal routes. A missing ID should lead to an explicit failure without allocating a copy or printing a delivered receipt. Alternatives earn credit when their operations and invariants meet the same needs.

Reflect: Which assumption did your answer depend on? Change that assumption, predict the consequence, then check it. Name the first step that would need repair.

Ready to move on: You can choose representations using workload, preserve identity across views and test component boundaries. Revisit the section where your trace first disagreed. The Test Bank chapter index supplies independent questions.

Glossary and retrieval cues

Adjacency list and matrix: Two ways to store edges. A list names each vertex's neighbours. A matrix records a value for each possible pair. [DS-CH07]

Array: An indexed sequence of slots. With equal-width slots, a known index locates a value directly. [DS-CH02]

Binary search tree: A binary tree with an ordering rule across entire subtrees. For distinct keys, smaller keys lie left and larger ones right. [DS-CH06]

Binary tree: A tree in which each node has up to two distinguished child positions, left and right. [DS-CH06]

Breadth-first search: An exploration that uses a queue to visit vertices by increasing edge count from the start. [DS-CH07]

Capacity: The number of reserved storage slots. A row can have capacity eight and contain only three current entries. [DS-CH02]

Child and parent: A directly lower node and the node immediately above it in a rooted tree. Nodes sharing a parent are siblings. [DS-CH06]

Cycle: A route along edges that returns to its starting vertex without repeating an interior vertex. [DS-CH07]

Depth and height: Depth counts edges from the root to a node. Height counts edges on the longest downward path from that node to a leaf. [DS-CH06]

Depth-first search: Exploration that follows an unvisited branch and backtracks when it runs out of choices. [DS-CH07]

Edge: A connection between vertices. It can carry a direction, a weight, both, or neither. [DS-CH07]

FIFO and LIFO: First in, first out describes a queue. Last in, first out describes a stack. [DS-CH04/05]

Graph: Vertices and the edges connecting them. A campus graph can contain alternate routes and cycles. [DS-CH07]

Head: A reference to the first node of a linked list. In an empty list head can be null. [DS-CH03]

Index: A position in an ordered sequence. With zero-based indexing the fourth entry has index 3. [ALG-CH04, DS-CH02]

Leaf: A tree node with no children. A one-node tree has a root that is also a leaf. [DS-CH06]

Length: The number of current entries. Valid indices of a row of length n are zero through n - 1. [DS-CH02]

Linked list: A sequence organized by references from node to node. Physical addresses need not follow sequence order. [DS-CH03]

Node: An individual element of a list, tree, graph or network. Its fields and connections depend on the structure. [DS-CH03, DS-CH06, AI-CH05/06]

Null: A marker meaning that no node is referenced. It differs from a node whose data value is zero. [DS-CH03]

Path: A sequence of vertices connected by edges, respecting direction. A simple path has no repeated vertex. [DS-CH07]

Peek and pop: Peek observes the top of a stack. Pop removes and returns it. Push adds a new top. [DS-CH04/05]

Preorder, inorder, postorder: Tree traversals that record a node before both subtrees, between left and right, or after both, respectively. [DS-CH06]

Queue: An arrival-ordered collection. Enqueue joins the rear and dequeue removes the front. [DS-CH04/05]

Record: A group of fields that belong together. A copy ID, title and shelf can describe one book. [ALG-CH01, DS-CH02]

Reference: Information that lets a process reach a particular node. A next reference names its successor. [DS-CH03]

Root: The starting node of a rooted tree. It has no parent within that tree. [DS-CH06]

Stack: A collection accessed at its top. The most recently pushed remaining item is the next one popped. [DS-CH04/05]

Subtree: One node together with all of its descendants in a rooted tree. [DS-CH06]

Traversal: Systematically visiting elements. A tree's level-order traversal visits by depth. [DS-CH02, DS-CH07]

Vertex and weight: A vertex is a graph node. A graph edge weight can be a travel cost; a neural weight multiplies an input contribution. [DS-CH07, AI-CH02/03/04]

Interface: The promise made by operations, separate from storage. A queue removes the oldest waiting item. [DS-CH01]

Invariant: A condition preserved by every permitted operation, such as arrival order among waiting queue entries. [DS-CH01]

Canonical record: The designated source of current facts that other views refer to. [DS-CH08]

Reading, sources and credits

The chapter structure follows the supplied three-volume Visual Edition V1: seven Algorithms chapters, eight Data Structures chapters and ten Connectionist AI chapters. The earlier project manuscript supplies revised stories and worked cases. This edition adds separate teaching sequences, new practice and a larger assessment companion. The page limit was relaxed by the user to allow teaching depth and visuals.

VisuAlgo: linked lists, stacks and queues

Steven Halim and the VisuAlgo team, National University of Singapore | DS-CH03 to DS-CH05

This combined visualizer lets you compare sequence links with restricted stack and queue operations.

https://visualgo.net/en/list

Data Structure Visualizations

David Galles, University of San Francisco | ALG-CH06; DS-CH04 to DS-CH07

The public index links array and linked implementations of stacks and queues, comparison sorting, search trees and graph traversals.

https://www.cs.usfca.edu/~galles/visualization/Algorithms.html

DSA Lab

DSA Lab project | ALG-CH03 to ALG-CH07; DS-CH01 to DS-CH08

The English index lists fundamentals, linear structures, trees, graphs and sorting with step-by-step material.

https://www.dsalab.dev/en

Open Data Structures: AdjacencyMatrix

Pat Morin | DS-CH07

A matrix representation turns graph connections into entries that can be inspected directly.

https://opendatastructures.org/ods-python/12_1_AdjacencyMatrix_Repres.html

Web review date: 6 November 2026. Public page review is distinguished from interactive execution in the Learning Materials companion. No external images or large source passages are reproduced. Technical diagrams are original project drawings; the anime illustrations were generated with the built-in image tool using the supplied visual direction. AI assistance supported writing, assessment, art and production.

Project evidence includes earlier source-review and verification records. The approved Markdown Content V2.0, a separate grading rubric and edition-specific instructor feedback were unavailable in that record and remain items for the team to reconcile. This publication does not claim instructor approval, independent student testing or a fresh full viewing of the existing video library. Editable source files and the verification report accompany the PDFs.